National Technical University of Athens
School of Electrical and Computer Engineering
Division of Industrial Electric Devices and
Decision Systems

Predictive Process Monitoring with Generative Artificial Intelligence

DIPLOMA THESIS

Apostolos Tzellas

Supervisor: Grigoris Mentzas, Professor, NTUA

Athens, June 2026

National Technical University of Athens
School of Electrical and Computer Engineering
Division of Industrial Electric Devices and
Decision Systems

Predictive Process Monitoring with Generative Artificial Intelligence

DIPLOMA THESIS

Apostolos Tzellas

Supervisor: Grigoris Mentzas, Professor, NTUA

Approved by the three-member examination committee on 10/07/2026

Grigoris Mentzas	Dimitrios Askounis	Evangelos Spiliotis
Professor, NTUA	Professor, NTUA	Assistant Professor, NTUA

Athens, June 2026

Apostolos Tzellas
Diploma in Electrical and Computer Engineering

Abstract

Predictive Process Monitoring aims to support operational decision-making by estimating the future behavior of ongoing process executions from event-log data. Among its central tasks, next-activity prediction focuses on identifying the activity that is most likely to occur immediately after the currently observed execution history of a running case. This task is important for runtime decision support, but remains challenging because real-world event logs often exhibit heterogeneous behavior, varying control-flow patterns, contextual attributes, and limited generalization across unseen process situations.

This diploma thesis investigates the use of Retrieval-Augmented Generation for next-activity prediction in business process event logs. The proposed framework represents process executions through structured textual encodings, retrieves similar historical examples from an indexed event-log memory, and provides them to Large Language Models as in-context demonstrations for predicting the next activity. In this way, the prediction is grounded in relevant historical process behavior, while the language model is used to interpret the retrieved context and generate the final answer.

The framework is evaluated on multiple real-life event logs from financial, administrative, healthcare, and billing-related domains. The experimental study examines the effect of different Large Language Models, encoding choices, embedding models, and an optional reranking stage on prediction quality. In addition, a more generalizable evaluation setting is considered in order to assess whether the proposed approach remains reliable beyond favorable experimental conditions. The results indicate that the RAG-based pipeline achieves promising performance for next-activity prediction and that its main components have a measurable impact on the final results. The findings further suggest that the proposed framework remains robust under a stricter evaluation protocol, highlighting the potential of retrieval-augmented in-context learning for predictive process monitoring.

Keywords: predictive process monitoring, next-activity prediction, retrieval-augmented generation, large language models, in-context learning, event logs, process mining

Acknowledgements

With the completion of this diploma thesis and the broader conclusion of my studies, I would like to express my sincere gratitude to everyone who contributed to and supported this effort.

First, I would like to warmly thank my parents and my grandmother for their unwavering support, encouragement, and love throughout my studies.

I would also like to thank my siblings, who were always by my side and helped me in every possible way.

My girlfriend was also a major source of support, standing beside me throughout this effort and helping me meaningfully complete my studies.

The guidance and support of Dr. Alexandros Bousdekis and PhD candidate Georgios Aristofanous were especially important; their advice, suggestions, and openness helped me grow as a researcher and shape this work in the best possible way. I would also like to sincerely thank the supervisor of my diploma thesis, Professor Grigoris Mentzas, for the trust he placed in me by assigning this topic and for his valuable guidance throughout this journey.

Contents

Chapter 1

Introduction

Business processes are the operational backbone of modern organizations. They describe how work is performed, how information moves between actors and systems, and how decisions are taken across domains such as finance, healthcare, logistics, public services, and enterprise management. As these processes are increasingly supported by information systems, their executions are recorded in the form of event logs. These logs provide detailed evidence of how process instances actually unfold, typically including information such as executed activities, timestamps, case identifiers, and additional event-level attributes [1, 2].

The availability of event-log data has made process mining an important research field for discovering, monitoring, analyzing, and improving real-world processes [1, 2]. Traditional process mining has often been used retrospectively: historical logs are examined to discover process models, check conformance, detect deviations, or identify bottlenecks after they have occurred. Although this kind of analysis is valuable, many operational environments require more than post-hoc understanding. Organizations increasingly need methods that can support decisions while a process instance is still running.

Predictive Process Monitoring addresses this need by focusing on the future behavior of ongoing process executions [1, 2]. Instead of only describing what has already happened, predictive monitoring aims to estimate what is likely to happen next, whether a case may lead to an undesired outcome, how much time remains until completion, or how the rest of the process may unfold. This thesis focuses on one of the central tasks within this area: *next-activity prediction*. Given the observed history of an ongoing case, the goal is to predict the activity that will immediately follow [3].

Next-activity prediction is practically important because it provides a fine-grained estimate of the near future of a running process instance. A reliable prediction of the next step can support operational planning, resource allocation, early warning mechanisms, compliance monitoring, and runtime decision support. It can also serve as a building block for broader predictive and prescriptive process monitoring systems, since many higher-level interventions depend on understanding the likely continuation of a case.

At the same time, next-activity prediction is challenging. Real event logs may contain noisy behavior, repeated patterns, rare variants, incomplete information, changing process conditions, and heterogeneous attributes. A predictive system must therefore be able to interpret partial process histories, connect them with relevant historical behavior, and produce useful predictions even when new cases do not exactly reproduce previously observed executions [3–5]. This makes the task not only a question of classification accuracy, but also a question of representation, contextualization, and generalization.

1.1 Research Context

Predictive Process Monitoring has been studied through a wide range of methodological approaches [1, 2]. Classical methods often rely on handcrafted feature representations, bucketing strategies, and conventional machine-learning classifiers. In such pipelines, the observed state of a running case is transformed into a structured representation, and a predictive model is trained to estimate a future property of the process [6, 7].

More recent work has explored deep-learning approaches, including recurrent neural networks, LSTMs, GRUs, convolutional models, attention mechanisms, Transformer-based

architectures, and graph neural networks [4, 5, 8–11]. These methods are particularly relevant for next-activity prediction because they can model the sequential nature of process executions more directly than traditional feature-based models. They have contributed significantly to the development of predictive process monitoring and remain strong reference points for the field.

However, existing approaches also involve methodological and practical limitations. Many predictive pipelines depend on fixed feature encodings, task-specific model training, and careful preprocessing choices. Their performance may be affected by how process behavior is represented, how event attributes are incorporated, how historical examples are used, and how the evaluation setting is constructed [3–5]. Moreover, many models operate as closed predictive components, making it difficult to dynamically use similar historical executions as explicit evidence during inference.

Large Language Models introduce a different perspective. Their ability to follow instructions, process textual and semi-structured inputs, and use examples provided in context makes them attractive for process-oriented tasks [12–18]. Instead of representing process behavior only through conventional numerical features, the observed history of a case can be expressed in a structured textual form and provided to a language model through a prompt. This creates the possibility of using LLMs not only as general-purpose text generators, but also as components in predictive process monitoring pipelines.

Nevertheless, using an LLM alone is not necessarily sufficient for process prediction. The future behavior of a process depends on the specific event log, the domain, and the historical regularities of the organization or system being analyzed. A language model may not know these regularities from its pretrained knowledge. Therefore, for predictive process monitoring, the model should be grounded in process-specific evidence.

Retrieval-Augmented Generation provides a way to achieve this grounding [19–21]. In a RAG-based setting, relevant information is first retrieved from an external collection and then inserted into the model context before generation. In this thesis, the external collection consists of historical process examples extracted from event logs. For each case to be predicted, similar historical examples are retrieved and provided to the LLM as contextual evidence. The model is then asked to use this evidence to infer the next activity.

This makes the proposed framework a retrieval-supported in-context learning approach for next-activity prediction [12, 19, 22]. The LLM receives task instructions, the current case representation, and relevant historical examples. The central idea is that retrieved examples can act as dynamic demonstrations of process behavior, allowing the model to make predictions that are grounded in the event log rather than relying only on general language-model knowledge.

1.2 Motivation

The motivation of this thesis comes from the convergence of three important needs.

First, organizations require predictive monitoring methods that can support runtime decisions. In many processes, knowing what is likely to happen next can be more useful than only analyzing what happened after completion. Next-activity prediction offers an immediate and operationally meaningful form of process prediction because it focuses on the next step of a running case.

Second, process behavior is difficult to represent fully with simple fixed encodings. Event logs contain ordered activity sequences, timestamps, lifecycle information, and event-level attributes. A useful predictive framework should preserve the sequential nature of the process while also allowing relevant contextual information to influence the prediction. This motivates the use of encoding methods that can represent process executions in a form suitable for both retrieval and language-model reasoning.

Third, recent progress in Large Language Models and Retrieval-Augmented Generation creates an opportunity to rethink next-activity prediction as an in-context learning problem

[12,15,19–21]. Historical process executions can be used not only for offline training or evaluation, but also as retrievable evidence during inference. This is especially appealing in process mining, where the event log itself contains the most relevant knowledge about how cases usually evolve.

The thesis is therefore motivated by the question of whether a RAG-based LLM pipeline can produce promising and reliable next-activity predictions when it is grounded in historical process behavior. It also investigates whether different parts of the pipeline, such as the LLM, the encoding strategy, the retrieval configuration, and the evaluation setting, meaningfully affect the final predictive performance.

1.3 Proposed Approach

The proposed framework uses a Retrieval-Augmented Generation pipeline for next-activity prediction in event logs. Raw event logs are first preprocessed and transformed into prediction examples. Each example represents the observed history of a case up to a specific point, while the target is the activity that follows.

The framework uses encoding methods to represent process executions in a form that can be used by both retrieval models and Large Language Models. On the retrieval side, such textual representations can be embedded and compared through dense similarity search using modern sentence-embedding models [23–26]. At the same time, the representations are designed to preserve the sequential behavior of the process and include relevant contextual information from the event log. In this way, process history can be used not only as structured data, but also as evidence that can be incorporated into a language-model prompt.

After encoding, historical examples are embedded and stored in a retrieval collection. For each case to be predicted, similar historical examples are retrieved using dense similarity search. These examples are then placed inside a structured prompt and provided to the LLM as in-context demonstrations. The model uses this retrieved context to return the predicted next activity in a controlled answer format.

The approach therefore combines process encoding, retrieval of similar historical behavior, and LLM-based in-context learning. The main idea is to ground next-activity prediction in relevant examples from the event log while using the LLM as the component that interprets the retrieved context and produces the final prediction.

1.4 Research Questions and Contributions

The contributions of this thesis are expressed through the four research questions that guide the experimental study. Each research question corresponds to a central part of the proposed framework and evaluates how that part affects the final prediction quality.

RQ1: How does the choice of large language model affect prediction quality in the proposed RAG-based next-activity prediction framework?

This question examines the role of the generator in the pipeline. Different LLMs may vary in their ability to follow instructions, interpret structured prompts, and use retrieved examples effectively. The contribution of this research question is to clarify how important the choice of language model is within a RAG-based predictive process monitoring framework.

RQ2: How do prefix-construction choices, particularly the gap and last-m parameters, influence the predictive performance of the proposed framework?

This question focuses on the way process histories are constructed and represented before prediction. Different construction choices can affect the amount, density, and type of information made available to the retrieval and generation stages. The contribution of this research question is to evaluate how such representation choices influence the quality and stability of the final predictions.

RQ3: What indications do the evaluated runs provide about the effect of retrieval-side choices, including the embedding model and the optional reranking step, on final prediction performance?

This question investigates the retrieval component of the framework. Since RAG depends on the relevance of the examples supplied to the LLM, the embedding model and reranking step may influence the usefulness of the retrieved context. The contribution of this research question is to assess how retrieval-side configuration choices affect the final predictive behavior of the system.

RQ4: How does a more reliable and generalizable evaluation setting affect the prediction quality of the framework?

This question addresses the evaluation strategy. Predictive process monitoring results can be overly optimistic when the experimental setting allows the model to benefit from conditions that are too favorable or not sufficiently representative of real deployment. A stricter generalized evaluation setting provides a more reliable view of how the framework behaves when it must generalize beyond the most familiar patterns in the available data. The contribution of this research question is to examine whether the proposed approach remains effective under a more realistic evaluation protocol.

Together, these research questions form the main contributions of the thesis. The thesis does not only propose a RAG-based framework for next-activity prediction, but also studies how its major components influence performance. In this sense, the work contributes an empirical analysis of an LLM-based predictive process monitoring pipeline, with attention to model selection, process representation, retrieval configuration, and evaluation reliability.

1.5 Summary of Findings

The experimental findings indicate that the proposed framework achieves promising results for next-activity prediction. Across the evaluated settings, the RAG-based pipeline shows that retrieved historical examples can provide useful in-context evidence for LLM-based prediction. This supports the central idea of the thesis: next-activity prediction can benefit from combining process-oriented representations with retrieval-augmented in-context learning.

The results also show that the evaluated components of the pipeline matter. The choice of language model affects prediction quality, indicating that the generator is not a neutral component. Encoding choices influence how process behavior is represented and made available to the model. Retrieval-side choices, including the embedding model and the use of reranking, also provide useful indications about how the quality of retrieved examples affects final predictions.

Importantly, the findings suggest that the proposed framework remains robust and reliable under a more generalized evaluation setting. This is particularly relevant because predictive process monitoring systems should not only perform well under favorable experimental conditions, but should also maintain useful behavior when evaluated under stricter conditions that better reflect generalization. The results of the generalized evaluation therefore strengthen the evidence that the proposed RAG-based framework is a viable direction for predictive process monitoring.

1.6 Scope and Limitations

The scope of this thesis is next-activity prediction in business process event logs. Other predictive process monitoring tasks, such as outcome prediction, remaining-time prediction, timestamp prediction, and full suffix prediction, are outside the main focus. This allows the study to examine one central prediction task in a focused and systematic way.

The proposed approach is evaluated as a retrieval-augmented prompting framework rather than as a fine-tuned task-specific LLM. The language model is used through prompts and in-context examples, while the event log provides process-specific grounding through retrieval. This design improves flexibility, but it also makes performance dependent on prompt construction, retrieval quality, and the model's ability to use the retrieved examples correctly.

Another limitation concerns the retrieval stage. Similarity in the retrieval space does not always guarantee process-relevant similarity. Two cases may appear similar according to the retrieval mechanism while differing in important behavioral details, and relevant historical behavior may not always be retrieved successfully. The optional reranking step is included to explore this issue, but retrieval quality remains a central challenge for RAG-based next-activity prediction.

Finally, evaluation in predictive process monitoring is sensitive to dataset characteristics, split strategy, repeated behavior, and metric choice. For this reason, the thesis includes a more generalized evaluation setting, but further validation across additional logs, domains, and experimental protocols would be valuable in future work.

1.7 Thesis Structure

The remainder of the thesis is organized as follows.

Chapter 2 presents the literature review and theoretical background. It introduces the main concepts used throughout the thesis, including events, traces, event logs, prefixes, predictive process monitoring, and next-activity prediction. It also discusses Large Language Models, Retrieval-Augmented Generation, embeddings, reranking, and related work on next-activity prediction.

Chapter 3 describes the methodology of the proposed framework. It presents the preprocessing steps, encoding and feature extraction, split strategies, retrieval layer, prompt construction, prediction flow, and evaluation process.

Chapter 4 presents the experimental setting and results. It describes the datasets, embedding models, reranker, LLMs, evaluation metrics, and experimental analysis corresponding to the four research questions.

Chapter 5 concludes the thesis. It summarizes the main findings, discusses limitations, and outlines possible directions for future work.

Chapter 2

Literature Review

2.1 Preliminaries

This section introduces the main concepts and notation used throughout the thesis. In particular, it formalizes event logs, traces, events, prefixes, and the predictive process monitoring setting considered in the experimental study.

2.1.1 Event

Let $\mathcal{A}$ denote the finite set of possible activity labels and let $\mathcal{T}$ denote the timestamp domain. For each event, additional recorded data values are represented by an attribute vector in a domain $\mathcal{V}$. An event is therefore defined as

$$e = (c, a, \tau, \mathbf{v}),$$

where c is the case identifier, $a \in \mathcal{A}$ is the executed activity, $\tau \in \mathcal{T}$ is the timestamp of occurrence, and $\mathbf{v} \in \mathcal{V}$ contains the event-associated attributes. The case identifier links the event to one concrete process instance, while the activity and timestamp determine what happened and when it happened.

2.1.2 Trace

A trace represents the observed execution history of one process instance. Formally, a trace is a finite ordered sequence of events

$$\sigma = \langle e_1, e_2, \ldots, e_n \rangle,$$

where all events in σ belong to the same case and are arranged chronologically. The length of the trace is denoted by $|\sigma| = n$. Since next-activity prediction depends on the order in which the process unfolds, preserving this temporal structure is essential.

2.1.3 Event Log

An event log is a collection of traces,

$$\mathcal{L} = \{\sigma_1, \sigma_2, \ldots, \sigma_N\},$$

where each trace corresponds to one completed or partially observed process instance. Operationally, the same control-flow pattern may appear in multiple cases, so the log can also be viewed as a multiset of traces. In either view, the event log is the empirical basis from which historical behavioral regularities are learned.

2.1.4 Prefix

Given a trace $\sigma = \langle e_1, e_2, \ldots, e_n \rangle$, a prefix of length t, with $1 \leq t < n$, is the truncated sequence

$$p_t = \langle e_1, e_2, \ldots, e_t \rangle.$$

The prefix captures the state of the running case after the first t observed events. In predictive process monitoring, prefixes are the natural prediction units because they represent incomplete executions for which the future is not yet known at prediction time.

2.1.5 Prefix Set

For a trace $\sigma = \langle e_1, e_2, \ldots, e_n \rangle$, the set of all prefixes can be written as

$$\text{Pref}(\sigma) = \{\langle e_1 \rangle, \langle e_1, e_2 \rangle, \ldots, \langle e_1, e_2, \ldots, e_{n-1} \rangle\}.$$

In next-activity prediction, these prefixes are the partial observations from which prediction instances are derived.

2.1.6 Predictive Process Monitoring Setting

The general predictive process monitoring setting can be described as follows. Starting from a historical event log $\mathcal{L}$, a model receives a running-case prefix p_t and must estimate some future property of the underlying trace, such as the remaining time, the final outcome, or the next executed activity. In abstract form, this can be written as a prediction function

$$f : \mathcal{P} \to \mathcal{Y},$$

where $\mathcal{P}$ is the space of admissible prefixes and $\mathcal{Y}$ is the target space associated with the predictive task under study.

2.1.7 Next-Activity Prediction

In this thesis, the task of interest is next-activity prediction. Let a_i denote the activity label of event e_i. For a prefix

$$p_t = \langle e_1, e_2, \ldots, e_t \rangle,$$

extracted from a longer trace $\sigma = \langle e_1, e_2, \ldots, e_t, e_{t+1}, \ldots, e_n \rangle$, the prediction target is the activity of the immediately following event:

$$y_t = a_{t+1}.$$

Accordingly, the learning problem is based on prefix–target pairs of the form

$$(p_t, y_t),$$

where the model must infer the most plausible next activity from the observed execution history. In the control-flow perspective, the target belongs to the activity alphabet $\mathcal{A}$. In richer settings, the decision may also be informed by event attributes, recent context, or retrieved historical examples.

2.2 Predictive Process Monitoring

Predictive Process Monitoring is a branch of process mining concerned with estimating the future evolution of an ongoing, not-yet-completed process execution [1, 2]. Typical prediction targets include the eventual outcome of a case, its remaining completion time, and the sequence or identity of future activities. In practical settings, such predictive capability is valuable because it enables organizations to anticipate delays, failures, or other undesirable developments before they fully materialize, thereby supporting earlier and more informed intervention.

Unlike purely reactive monitoring [27], which identifies a problem only after it has already occurred, predictive monitoring aims to detect risk in advance so that preventive actions can still be taken. For this reason, the area has become an important research direction at the intersection of process mining, predictive analytics, and data-driven artificial intelligence. Its relevance is also strongly practical, since predictive functionalities can be incorporated into operational environments where organizations seek decision support for running cases rather than post-hoc analysis alone.

Most Predictive Process Monitoring approaches rely on historical completed executions in order to produce estimates for the future of an incomplete running case. In broad terms, they usually involve two phases. First, in an offline learning phase, a predictive model is derived from historical traces. Then, in an online runtime phase, that learned model is queried with an observed prefix of an ongoing case in order to generate a prediction about its future behavior.

The literature further emphasizes that Predictive Process Monitoring is not a single prediction problem, but rather a family of runtime decision-support tasks defined over ongoing cases [1, 2]. Some tasks are categorical, such as predicting whether a case will end in success, violation, or escalation. Others are numerical, such as forecasting remaining processing time, future timestamps, or workload-related indicators. A further class is sequence-oriented, where the objective is to estimate the next activity or even a longer continuation suffix of the running trace. This broader view is important because it shows that the field is organized less around one specific model family than around a common operational question: given the partial history of an execution, what can be inferred about its future quickly enough to support intervention? Review work on outcome-oriented monitoring and later conceptual surveys both present the field in precisely this runtime-oriented way, with the prefix of a running case serving as the central predictive unit [1, 2, 6].

From a methodological perspective, many PPM pipelines share a common structure even when their learning algorithms differ. Historical event logs are first transformed into prefix-based instances, because predictions must be made on incomplete traces rather than on completed cases. These prefixes are then encoded so that both the executed control-flow and, where useful, the accompanying event data can be exploited by the predictor. In early frameworks such as clustering-based predictive monitoring, prefixes are grouped according to behavioral similarity and then associated with specialized classifiers that use event attributes to discriminate among possible future outcomes [7]. Later overviews describe this general pattern in broader terms: historical traces provide the offline knowledge base, prefixes act as online query states, and the predictive model must connect the observed partial execution with a target defined over the unknown future of the case [1, 2]. In this sense, PPM can be understood as a runtime inference layer built on top of event-log history.

The cited literature also stresses that the main challenges of PPM are not only algorithmic but also methodological and operational. One recurring issue is comparability. Teinemaa et al. observe that studies often differ in datasets, preprocessing, evaluation measures, and experimental setup, which makes it difficult to judge the relative strengths of competing methods fairly [6]. More recent work extends this concern by arguing that predictive models should be evaluated not only on raw accuracy but also on their ability to generalize beyond repeated or overly favorable historical patterns [3]. Another challenge is environmental variability: real processes evolve over time, exhibit drift, and may generate new behavioral variants, which limits the usefulness of a static model trained once on past traces [2, 28]. For this reason, recent PPM research increasingly highlights robustness, adaptability, and experimental realism as core concerns alongside raw predictive performance.

A further implication of these studies is that useful PPM systems must balance predictive power with practical usability. In operational settings, predictions are expected to be timely, understandable, and actionable, not merely statistically strong in hindsight [2]. This requirement explains why the field has gradually expanded from isolated predictive models toward broader monitoring pipelines that also consider data representation, update mechanisms, evaluation design, and user trust. Within this broader perspective, next-activity prediction occupies a particularly important place: it is both a concrete prediction task in its own right and a fine-grained probe of whether a monitoring system can correctly interpret the evolving local state of a process case. This is the precise angle from which the present thesis approaches Predictive Process Monitoring.

2.3 Large Language Models

Large Language Models (LLMs) have emerged as flexible models that can act on natural-language instructions, integrate evidence from multiple inputs, and adapt to new tasks through prompt context rather than task-specific retraining [12]. This makes them relevant to process-oriented settings, where useful information is often scattered across activity labels, event attributes, textual descriptions, intermediate reports, and analyst questions. Compared with conventional predictive pipelines that depend on a fixed feature encoding and a separately trained downstream predictor, LLMs provide a more general mechanism for operating over textual and semi-structured process information.

In the broader process mining literature, LLMs have already been explored for tasks that go well beyond generic text generation. Berti and Qafari [13] show how LLMs can be used to query and interpret process mining artifacts through prompting, while Vidgof et al. [14] discuss their wider potential across business process management tasks and decision support scenarios. Related work has also studied LLMs as interfaces for answering process mining questions in natural language, as aids for generating or interpreting process models, and as general-purpose assistants for BPM tasks that would otherwise require more specialized tools or formal query languages [15–17]. Taken together, these studies indicate that LLMs can function not only as conversational front ends, but also as adaptable layers between analyst requests and process artifacts.

More directly related to predictive and prescriptive settings, recent studies show that LLMs can contribute to process mining in two complementary ways. First, they can support analyst-facing reasoning tasks. For example, Lashkevich et al. [18] use GPT-based prompting to examine waiting-time causes and suggest redesign alternatives, showing that structured prompts and explicit contextual inputs can make the generated recommendations more useful for process improvement. Second, LLMs can be used as semantic enrichment components inside a predictive pipeline instead of being treated as standalone predictors. Yuan et al. [29] use LLaMA-3 to derive business objects and status information from activity labels, and then feed that semantic layer into predictive process monitoring pipelines for outcome-oriented and next-activity prediction, reporting gains in downstream performance across many settings.

These studies also highlight an important methodological point. In process mining, LLMs tend to be more reliable when they are anchored by task-specific instructions, structured inputs, and carefully chosen examples, because process semantics depend heavily on domain context and cannot always be inferred robustly from pretrained knowledge alone [18, 29]. Accordingly, the role of the LLM is often augmentative: it helps interpret, organize, or contextualize process information that then supports analysis or prediction, rather than replacing the whole predictive pipeline with a single black-box component.

This observation leads naturally to in-context learning (ICL), which is especially relevant to the present thesis. In ICL, the model is steered at inference time through instructions and examples placed directly in the prompt, without parameter updates [12]. In the process domain, Yuan et al. [29] report that few-shot prompting yielded more stable semantic extraction behavior than weaker prompting settings, which motivated their use of multiple in-context examples during extraction. For predictive process monitoring, this matters because historical prefixes can themselves serve as task-specific demonstrations: they illustrate how partially observed execution histories relate to future outcomes or next activities. From this viewpoint, retrieval-augmented prompting can be seen as a practical realization of ICL with dynamically selected process-specific examples, which forms the conceptual bridge to the RAG-based approach discussed next.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines a generator with an external retrieval component so that the model does not rely exclusively on knowledge stored in its parameters [19]. At inference time, the input is first used to retrieve a small set of relevant documents, passages, or memory items from an indexed collection. These retrieved items are then inserted into the model context and used during response generation. This standard indexing–retrieval–generation pattern is also emphasized in later survey work on RAG for large language models [20]. In this way, generation is conditioned not only on the prompt itself, but also on explicit evidence selected from an external memory. Conceptually, this creates a hybrid architecture in which parametric knowledge from the language model is complemented by non-parametric knowledge that can be searched, inspected, replaced, or expanded without retraining the whole model.

This design is especially useful for knowledge-intensive tasks, where accurate output depends on access to information that is too detailed, too dynamic, or too extensive to be handled reliably through parametric memory alone. In such settings, a purely parametric model may answer fluently while still omitting relevant evidence, relying on outdated information, or hallucinating unsupported details. By grounding the generation step in retrieved evidence, RAG improves factuality and specificity while also making the source of the response more transparent. The broader RAG literature further shows that this pattern has become attractive because it allows language models to operate over domain collections without requiring repeated full-model updates whenever the underlying knowledge changes [21].

For the present thesis, the value of RAG lies in the fact that predictive process monitoring can also be viewed as a knowledge-intensive task, since the interpretation of an ongoing prefix may depend on recalling relevant historical precedents rather than only on generic language knowledge. Instead of asking the model to infer the next activity from the current prefix in isolation, a retrieval-based setup can supply similar previously observed prefixes together with their continuations, thereby grounding the prediction in concrete process history. This makes RAG a natural fit for settings where useful context is distributed across many past cases and where prediction quality may benefit from dynamically selecting the most relevant examples for the case at hand.

2.5 Embeddings and Reranking

Dense text embeddings have become a central component of modern retrieval systems because they map variable-length texts to fixed-size vectors whose relative positions reflect semantic relatedness. Recent progress has moved the field from general-purpose sentence encoders toward retrieval-oriented models trained and evaluated across broader task suites, including retrieval, clustering, pair classification, and reranking. The MTEB benchmark highlights both the breadth of current embedding applications and the fact that no single embedding approach dominates all tasks, which has motivated the development of more specialized retrieval encoders [23]. At the same time, recent families such as BGE show how embedding research has increasingly focused on retrieval quality, multilinguality, and robustness across different text granularities [24].

In practical dense retrieval, the dominant design is a bi-encoder setup: the query and each candidate text are encoded independently, and nearest neighbors are then selected through a similarity measure such as cosine similarity. This design is attractive because document embeddings can be precomputed and indexed once, while only the query embedding must be produced online. Sentence-BERT made this paradigm especially influential for semantic search by adapting BERT into a siamese and triplet architecture that yields sentence embeddings directly, greatly improving the efficiency of large-scale similarity search compared with pairwise cross-encoding [25]. More lightweight variants, including MiniLM-based sentence encoders, showed that strong semantic retrieval behavior can also be achieved with smaller distilled

transformer models [26]. Closely related encoders have also been used as practical retrieval baselines in recent RAG-based next-activity prediction work [22].

Dense retrieval, however, gains efficiency by giving up fine-grained query–document interaction at ranking time. For this reason, many RAG pipelines add a second-stage reranker after initial retrieval. Recent RAG surveys describe reranking as a post-retrieval refinement step that reorders a small candidate pool so that the most relevant chunks are promoted into the final context window [20]. This follows the cross-encoder reranking paradigm popularized by BERT-based passage reranking, where the query and candidate passage are scored jointly rather than through independent embeddings [30]. Although such models are more expensive than bi-encoders, they have shown strong gains in early-rank retrieval quality and are therefore commonly used to refine a limited candidate pool before final generation. In this way, reranking complements dense retrieval rather than replacing it, serving as a targeted refinement stage within retrieval-augmented pipelines.

2.6 Related Work

Next-activity prediction is one of the central tasks in Predictive Process Monitoring. Given a partially observed prefix of an ongoing execution, the objective is to estimate the activity that is most likely to occur next. This task belongs to the broader family of predictive process monitoring problems, which also includes outcome prediction and time-oriented targets such as remaining time and timestamp estimation [1, 2]. In most predictive pipelines, historical event logs are transformed into prefix-based training or inference instances, and the resulting representations are then used by a downstream predictor at runtime [2]. Existing next-activity prediction approaches can therefore be organized according to the way prefixes are represented and the type of predictive model that consumes them.

One important line of work is based on process-aware and feature-based machine learning. In these approaches, prefixes are encoded as structured feature vectors, sometimes after applying bucketing strategies such as prefix-length grouping, clustering, or state-based partitioning [2]. The prediction task is then handled with conventional supervised-learning models such as decision trees, random forests, support vector machines, gradient-boosting methods, or ensembles. Although these methods are less expressive than more recent neural architectures, they remain relevant because they offer strong baselines and are often easier to interpret and analyze. A representative recent example is the AutoML-based framework of Kaftantzis et al. [31], which automates pipeline selection and hyperparameter optimization for next-activity prediction.

A highly influential research direction is based on deep sequence learning. Here, the prefix is modeled explicitly as an ordered event sequence, and temporal dependencies are learned through recurrent neural networks, long short-term memory networks, gated recurrent units, convolutional architectures, or encoder–decoder designs. Recent review and benchmark studies show that deep-learning approaches have become a dominant direction in predictive business process monitoring because they can capture control-flow regularities directly from sequential histories and often outperform more traditional feature-based baselines [4, 5]. At the same time, this family remains methodologically diverse, since performance is strongly influenced by preprocessing decisions, encoding choices, architectural design, and the particular prediction target under consideration [4].

Beyond purely sequential modeling, another stream of research extends next-activity prediction with data-aware, multi-perspective, and explainable mechanisms. Instead of relying only on activity labels, these methods incorporate additional process perspectives such as timestamps, resources, categorical attributes, numerical values, or case-level context. Multi-view approaches, for instance, learn separate representations for different perspectives of the same execution and then combine them into a unified predictor [8]. Other work focuses on

explainability. DENAP, for example, combines deep next-activity prediction with Layer-Wise Relevance Propagation in order to identify which previous activities and data attributes were most influential for the predicted next step [32]. This direction is consistent with the wider demand for interpretable and explainable predictive process monitoring models, especially in operational settings where predictions are expected to support human decisions [33].

Further architectural developments include attention-based, Transformer-based, and graph-based methods. Transformer architectures are attractive because attention mechanisms can model longer-range dependencies more directly than purely recurrent designs. Recent examples include JARVIS, which combines multi-view trace representations with Vision Transformers and adversarial training for next-activity prediction [34], and the Switch-Transformer approach of Nguyen [9], which adopts a mixture-of-experts Transformer architecture for next process-activity prediction. In parallel, graph-based methods represent traces, process relations, or event dependencies as graphs. Chiorrini et al. [10] use multi-perspective enriched instance graphs together with graph neural networks, while Rama-Maneiro et al. [35] combine graph convolutional networks with recurrent models to exploit both event-log and process-model information. More recently, PROPHET has shown how heterogeneous graph neural networks can support both accurate and explainable next-activity prediction [11].

The most recent line of work moves toward semantically enriched trace representations and LLM-based reasoning. In these approaches, prefixes are reformulated as textual or semantically structured inputs, and the prediction is inferred through prompting, in-context examples, or retrieval-supported generation rather than through a task-specific classifier alone. Within this direction, Retrieval-Augmented Generation is especially relevant because it allows the model to ground the current prediction in similar historical prefixes retrieved from an indexed memory. In RAG-based next-activity prediction, historical prefixes are embedded, stored in a vector index, and retrieved according to their similarity to the running case; the retrieved examples are then injected into the prompt as contextual evidence for the final prediction [22]. This shifts the task away from a purely supervised sequence-classification formulation toward a retrieval-supported reasoning setting in which predictions are explicitly informed by analogous past executions.

Taken together, these developments motivate the central perspective of the present thesis. Building on the promising use of LLMs in predictive process monitoring, and on retrieval pipelines that combine textual prefix encoding with dense retrieval and reranking mechanisms discussed in the literature [20, 22, 30], this thesis examines how such techniques behave when integrated into a RAG-based next-activity prediction framework. In this sense, the study extends prior work on retrieval-augmented next-activity prediction by systematically evaluating alternative encoding and retrieval configurations, while also investigating the effect of using LLMs of different sizes within the same overall pipeline.

Chapter 3

Methodology

This chapter describes the workflow followed to produce next-activity predictions, from the raw event log to the final evaluation stage. The pipeline includes:

- **Data preprocessing.** At this stage, raw XES event logs are parsed and converted into a structured format suitable for downstream processing. Cleaning and filtering steps are then applied so that only the events and attributes that are useful for prediction are retained.
- **Encoding and feature extraction.** Each trace is transformed into a collection of prefixes and corresponding target labels, where the label is the next activity to be predicted. In addition, selected information from recent events is preserved so that the model can exploit both control-flow and relevant attribute context.
- **Split strategy.** The generated examples are separated into retrieval and test collections according to the selected experimental protocol. Depending on the chosen mode, the split may be applied either at the prefix level or at the trace level before the final datasets are produced.
- **Retrieval of relevant historical examples.** For each query prefix, similar historical prefixes are retrieved from the training set using embeddings and vector search. The goal of this stage is to provide the language model with context from previously observed process executions.
- **Prompt construction for the language model.** The query prefix, the retrieved examples, and the required instructions are formatted into a unified prompt. In this way, the language model receives a structured input that guides it toward predicting the next activity.
- **Generation of the next-activity prediction.** The language model processes the final prompt and returns the predicted next activity for the examined prefix. This output is then isolated and converted into the final form used by the evaluation stage.
- **Performance evaluation.** The generated predictions are compared against the true next activities of the test set. Appropriate metrics are then used to assess the predictive quality of the system under different experimental settings.

Figure 3.1 summarizes the overall data flow of the proposed methodology, from raw event-log processing and split creation to retrieval, prompt assembly, next-activity prediction, and final evaluation. The diagram is intended as a high-level overview of the end-to-end workflow rather than as a one-to-one representation of every internal implementation step.

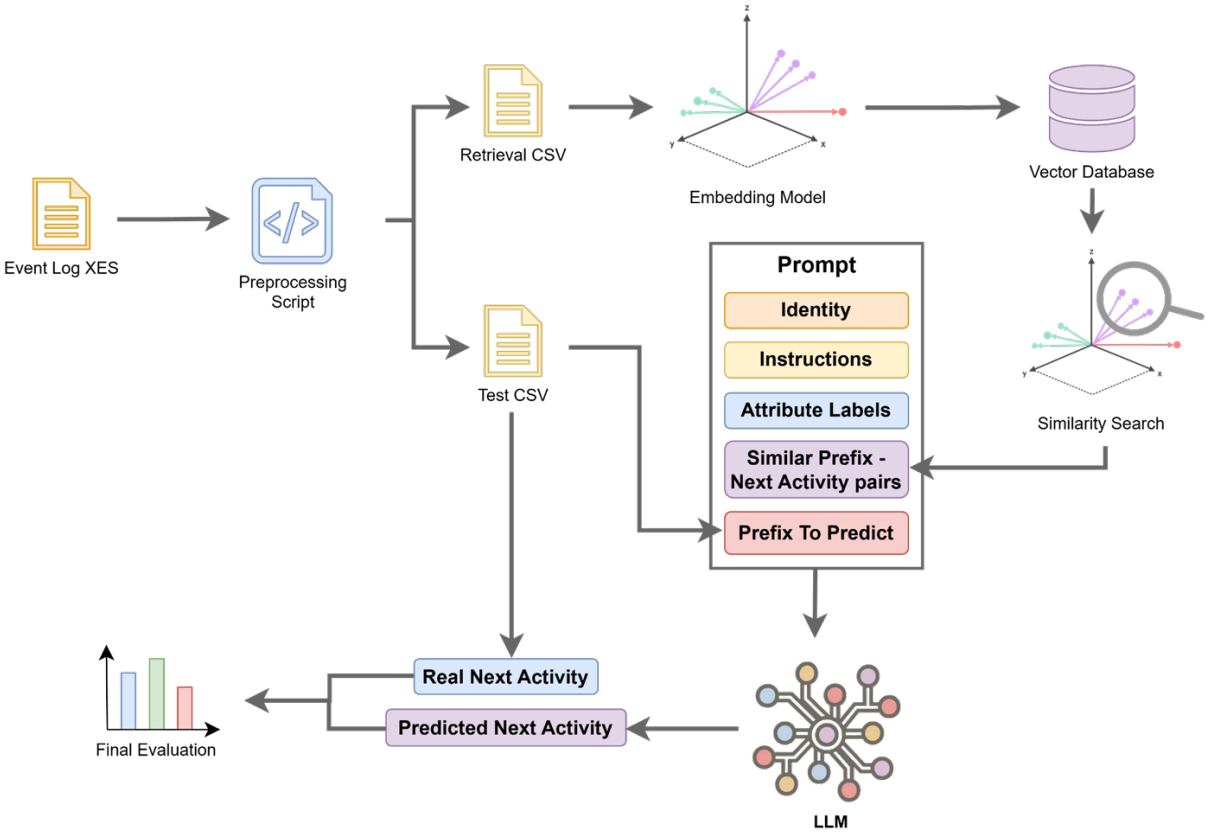

Figure 3.1: High-level overview of the full pipeline and evaluation flow

3.1 Data Preprocessing

Preprocessing starts from the original event log stored in XES format and converts it into a tabular representation in which each row corresponds to a single event of a process execution. At this stage, only the basic information required to reconstruct the event sequence of each case is preserved, namely the execution identifier, the activity name, the timestamp, and the available event-level attributes.

An initial cleaning step is then applied to remove records that are not suitable for the prediction setting. When lifecycle information is available in the log, only the events that indicate that an activity has been completed are retained. This ensures that each activity is represented once as an actual process step and prevents intermediate or auxiliary lifecycle transitions from distorting the execution sequence.

The next step restricts the set of attributes that are propagated to the later stages. Trace-level attributes that describe the execution as a whole are removed, with the exception of the trace identifier that is required to group events into the same trace. In contrast, event-level attributes that accompany each activity are preserved. This makes the dataset more consistent, avoids redundant information, and ensures that subsequent processing focuses mainly on the dynamic behavior of the process.

After cleaning, events are grouped by trace and arranged in the correct chronological order. This step is critical because next-activity prediction assumes an accurate representation of the path followed so far by each trace. For this reason, the sequence of events is reconstructed according to the available timestamps and, where necessary, the original appearance order is retained to resolve ties without altering the actual trace structure.

The outcome of this stage is not yet a learning-ready feature set, but a clean, consistent, and chronologically organized body of data that can support the subsequent construction of prefixes

and prediction targets.

Example before and after preprocessing

```
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_SUBMITTED"/>
  <date key="time:timestamp" value="2011-10-01T00:38:44.546+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_PARTLYSUBMITTED"/>
  <date key="time:timestamp" value="2011-10-01T00:38:44.880+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="SCHEDULE"/>
  <string key="concept:name" value="W_Completeren aanvraag"/>
  <date key="time:timestamp" value="2011-10-01T00:39:38.875+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="START"/>
  <string key="concept:name" value="W_Completeren aanvraag"/>
  <date key="time:timestamp" value="2011-10-01T11:36:46.437+02:00"/>
</event>
<event>
  <string key="lifecycle:transition" value="COMPLETE"/>
  <string key="concept:name" value="A_ACCEPTED"/>
  <date key="time:timestamp" value="2011-10-01T11:42:43.308+02:00"/>
</event>
```

Listing 3.1: Raw trace fragment from the original XES log

Table 3.1: Dataframe after preprocessing

trace_id	activity	timestamp	lifecycle
173688	A_SUBMITTED	2011-10-01 00:38:44.546+02:00	COMPLETE
173688	A_PARTLYSUBMITTED	2011-10-01 00:38:44.880+02:00	COMPLETE
173688	A_PREACCEPTED	2011-10-01 00:39:37.906+02:00	COMPLETE
173688	A_ACCEPTED	2011-10-01 11:42:43.308+02:00	COMPLETE

The example illustrates an indicative trace fragment from the original XES log and its corresponding form after preprocessing. In the raw version, each event contains lifecycle information and timestamps, whereas after preprocessing only completed activities and the core attributes required by the later stages are retained.

3.2 Encoding and Feature Extraction

After preprocessing, each clean trace is transformed into a set of training examples suitable for next-activity prediction. The key idea is that a complete trace is not used as a single sample. Instead, it is decomposed into multiple prefixes. Each prefix represents the execution state of the process up to a specific point, while the target label is defined as the immediately following activity. In this way, the original event log is converted into a collection of prefix–target pairs that can be used both for retrieval and for final prediction.

Feature construction is based on two complementary sources of information. The first is the *control-flow*, namely the sequence of activities that has already been executed in the trace. The second is the *data payload*, which contains the values of the attributes associated with the individual events. In the proposed pipeline, these two aspects are not encoded as a conventional numerical feature vector. Instead, they are transformed into a structured textual representation, where the prefix preserves the sequential activity structure while also including selected attribute values from the most recent events.

Formally, let a trace be denoted by

$$\sigma = \langle e_1, e_2, \ldots, e_n \rangle,$$

where e_i is the i-th event of the case. For any eligible position $t < n$, the corresponding prefix is

$$p_t = \langle e_1, e_2, \ldots, e_t \rangle,$$

and the prediction target is the activity of the immediately following event, denoted by

$$y_t = a_{t+1}.$$

The generation of prefixes is controlled by the *base* parameter, which defines the minimum length from which examples start to be created. This avoids generating excessively short prefixes, which usually contain limited information and are not sufficiently representative of the actual state of the case. For each eligible point in a trace, the sequence of activities up to that point is used as the observed history, while the immediately following activity is used as the correct answer to be predicted.

The *gap* parameter determines the spacing between consecutively generated prefixes. Instead of creating a new example after every next event, the pipeline can construct prefixes at sparser intervals. This reduces the density of almost identical examples and controls the size of the resulting dataset without removing the essential ordering information. The use of gap-based prefix generation follows a design logic already discussed in predictive process monitoring literature [2,7]. In the final evaluation, this parameter is treated as one of the main experimental levers because it directly affects how finely the evolution of each trace is observed.

Taken together, *base* and *gap* define the set of eligible prefix lengths. If $base = b$ and $gap = g$, these lengths can be written as

$$\ell_k = b + kg, \qquad k = 0, 1, 2, \ldots,$$

subject to $\ell_k < n$, where n is the length of the trace. The generated examples therefore take the form

$$(p_{\ell_k}, y_{\ell_k}).$$

This parameterization offers a simple way to control dataset granularity. Larger gap values reduce the number of generated examples and therefore help limit the size of the final dataset, but they also make the observation of the trace evolution coarser, since some intermediate states are no longer explicitly represented.

Last-state encoding was employed, as done in prior work on predictive process monitoring [28], with its extension to last-m-states encoding [6]. Within the present implementation, the m parameter determines how much recent attribute information is retained inside each prefix. The full activity sequence always remains visible, but detailed *value blocks* are preserved only for the last m events. As a result, the model retains access to the complete control-flow history, while only a limited and recent attribute context is kept in explicit form. In this way, the representation preserves the sequential evolution of the trace while focusing the detailed data payload on the most recent part of the execution, where attribute values are more likely to be directly relevant for the next prediction step.

If a prefix ends at position t, the control-flow history remains fully visible, while explicit attribute-value blocks are retained only for the events whose indices belong to the set

$$\{i \mid \max(1, t - m + 1) \leq i \leq t\}.$$

Thus, the representation keeps the full sequential history but restricts the detailed data payload to the most recent m events.

These values are not stored as isolated independent fields. Instead, they form a snapshot of the most recently available attribute state up to the examined point. In other words, each new event updates the current attribute picture, and only the most recent relevant snapshots remain visible in the final representation. In addition, attribute names are compacted into shorter identifiers so that the prefix remains concise and usable in the later stages of the pipeline without losing its informational content.

Finally, deduplication is applied at the level of the prefix control flow, so that the same activity sequence is not stored multiple times even if it appears with different repeated attribute realizations. In line with next-activity prediction literature that emphasizes the importance of testing on unseen prefixes in order to obtain meaningful conclusions about generalization [3], this choice reduces redundancy and helps prevent overly optimistic evaluation conditions caused by repeatedly exposing the model to the same behavioral history. The outcome of this stage is an organized set of textual prefixes that captures both the execution path of each trace and the most critical recent attribute context, as demonstrated in Listing 3.2.

```
event_A {Values},event_B {Values}
event_A,event_B,event_C {Values},event_D {Values},event_E {Values}
event_A,event_B,event_C,event_D,event_E,event_F {Values},event_G {Values},event_H {Values}
...
```

Listing 3.2: Indicative prefix representation

The above example shows successive prefixes following the same representation logic. For *base*=1, the first prefix is produced once the first two events have been observed. For *gap*=3, each subsequent prefix is extended by three new events. Finally, for m=3, detailed *Values* are retained only for the three most recent events, whereas older activities remain visible only as part of the *control-flow*.

3.3 Split Strategy

After feature construction, the pipeline can create the retrieval and test collections in two different ways. The difference lies in *when* the split is applied and in *which unit* is being separated. In both cases, deduplication follows the control-flow representation of the prefix, meaning that prefixes are considered duplicates when they express the same sequence of activities regardless of their attached value blocks. This design is aligned with recent next-activity prediction literature showing that evaluations dominated by repeated prefixes can obscure the actual generalization ability of prediction models [3].

3.3.1 Prefix Level

In the first mode, the full event log is processed as a single body of data. All traces are first converted into one global table of prefix–target pairs. At this stage, global deduplication is already active, so repeated control-flow prefixes are removed before the final split is created. As a result, the full prefix table contains only one retained copy of each unique behavioral history under the implemented deduplication rule.

Once this globally deduplicated table has been produced, the split is applied directly at the prefix level. In the current implementation, 30% of the rows are sampled from the table

to form the test set, while the remaining 70% form the retrieval set. In this configuration, the unit of separation is therefore the individual prefix row rather than the original trace. Since the split happens after all prefixes have already been generated, prefixes originating from the same original trace may end up on both sides, provided that they are not removed by the global deduplication step.

This mode is retrieval-oriented. It evaluates the system in a setting where the retrieval collection and the test collection are both derived from the same globally generated prefix space, but exact duplicates according to the control-flow deduplication key are removed before row sampling takes place. This split is also in line with prior work as described in [22].

3.3.2 Trace Level

In the second mode, the split is applied before prefix generation. More specifically, this is a *temporal split* performed at the trace level. The original XES log is first divided into two valid XES files: a retrieval-side file and a test-side file. In the current implementation, the traces are ordered chronologically according to their end timestamps, the earliest 70% of traces are assigned to retrieval, and the remaining later 30% are reserved for testing. In this way, the retrieval collection is built only from earlier completed cases, while the evaluation is carried out on later completed cases.

After these split files are created, each side is processed through the same preprocessing and prefix-construction pipeline. In the implemented trace- level setting, the same chronological trace split is kept, but a shared global deduplication state is used across retrieval and test during prefix generation. This means that once a control-flow prefix has already been retained on the retrieval side, the same prefix is not stored again on the test side. Because the retrieval side is processed first, the retained unique copy remains in retrieval and is excluded from the later test collection.

This design helps avoid data leakage by separating the data at the trace level rather than at the prefix level. Under a prefix-level split, different prefixes originating from the same original case may still appear on both sides of the partition, which means that the test set can contain behavioral histories that are only partial continuations of traces already represented in retrieval. By contrast, the temporal trace-level split ensures that complete later cases are held out from the retrieval memory, making the evaluation stricter, more realistic, and better aligned with the goal of assessing generalization to unseen future executions [3]. Further investigation of generalization is provided in Chapter 5.

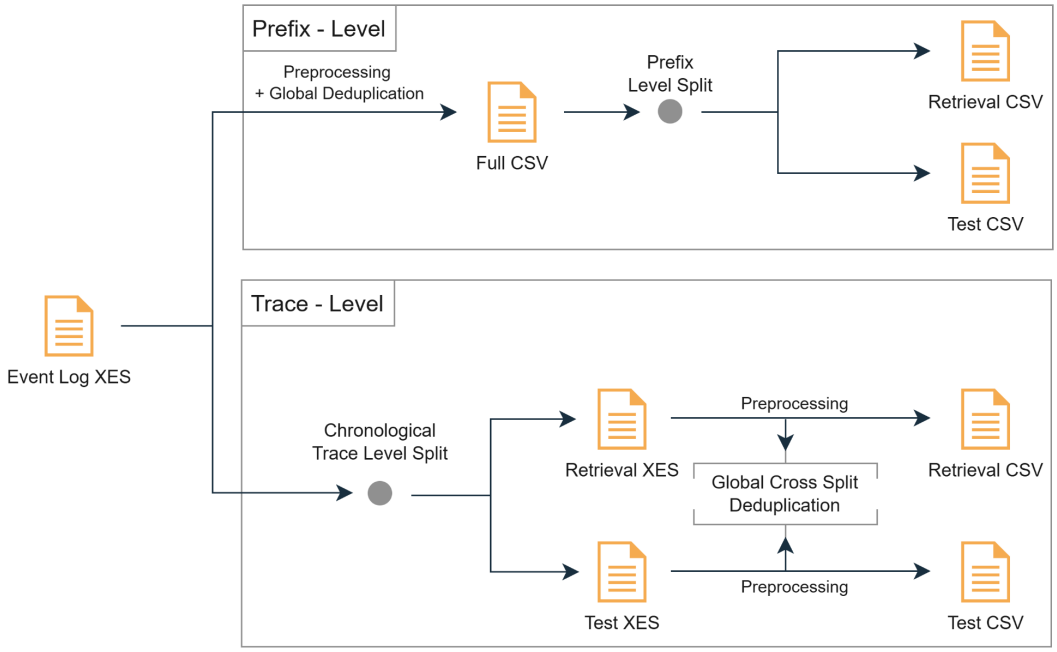

Figure 3.2: Split strategy pipeline

3.4 Retrieval Layer

Once the retrieval and test CSV files have been created, the next stage of the pipeline builds a searchable memory over the historical prefixes contained in the retrieval set. Each row of the retrieval CSV consists of a textual prefix and its corresponding next-activity label. The prefix is treated as the retrievable object, while the label is stored together with it so that, once a similar historical example is found, its known continuation can be passed to the prediction stage as contextual evidence.

To make this search possible, each retrieval prefix is transformed into a dense vector representation through an embedding model. In this way, prefixes that are similar in sequential structure and recent attribute context are mapped to nearby points in the vector space. After all retrieval prefixes have been encoded, their vector representations are stored together with the original textual prefix and the associated target label. As a result, the retrieval layer preserves both the compact numerical representation needed for similarity search and the original symbolic information needed for prompt construction.

During evaluation, each query prefix taken from the test CSV is encoded with the same embedding model. The resulting query vector is then compared against the stored retrieval vectors through similarity search, using a cosine-based comparison in the embedding space. The pipeline returns the top k most similar historical prefixes, where k is a configurable parameter of the experiment. Each retrieved item therefore contains three elements: the historical prefix itself, its known next-activity label, and its similarity score with respect to the query prefix. The role of this step is to provide a compact set of highly relevant past examples rather than the full retrieval collection.

An optional second ranking step can also be applied after the initial vector search. In this variant, a larger candidate pool is first collected by similarity search and then reordered according to a learned pairwise relevance criterion between the query prefix and each candidate prefix. The final top k examples are then selected from this reordered list. Whether this second ranking stage is used or not, the retrieval layer always produces the same type of output: a small ordered set of historical prefix–label examples that are passed to the prompt construction stage.

3.5 Prompt Construction

After the retrieval stage, the selected historical examples must be converted into a form that can be directly consumed by the language model. For this reason, the pipeline does not pass the query prefix alone. Instead, it builds a structured prompt that combines task instructions, contextual information, and the prefix whose next activity must be predicted. In the experimental setting, this contextual evidence is formed from the final top-k retrieved examples returned by the retrieval layer, with $k = 10$ in the main evaluations.

The prompt is organized into a small number of clearly separated blocks, as shown in Figure 3.3. First, an *identity* block specifies that the model acts as a process-mining assistant specialized in predictive monitoring. Second, an *instruction* block defines the task itself, namely to predict the next activity immediately after the last activity of the given prefix and to return the answer in a strictly controlled format. Third, the prompt may include an *attribute legend*, which explains the compact attribute identifiers used in the textual prefix representation. Finally, the prompt includes the retrieved historical examples, each represented as a past prefix together with its known next activity.

After these contextual sections, the target prefix is appended as the final query to be solved by the model. In this way, the model receives both the current process state and a compact memory of similar historical continuations. The retrieved examples therefore function as in-context evidence, while the query prefix defines the specific prediction instance of interest.

Two prompt variants are used in the experiments. The first is a general retrieval-based prompt, which instructs the model to reason from the retrieved past traces and the values of the most recent events. The second follows the same overall structure, but places stronger emphasis on the most recent attribute values retained by the *last-m* representation. The two variants therefore differ less in layout than in the weight they assign to the latest visible attribute context when predicting the continuation of the prefix.

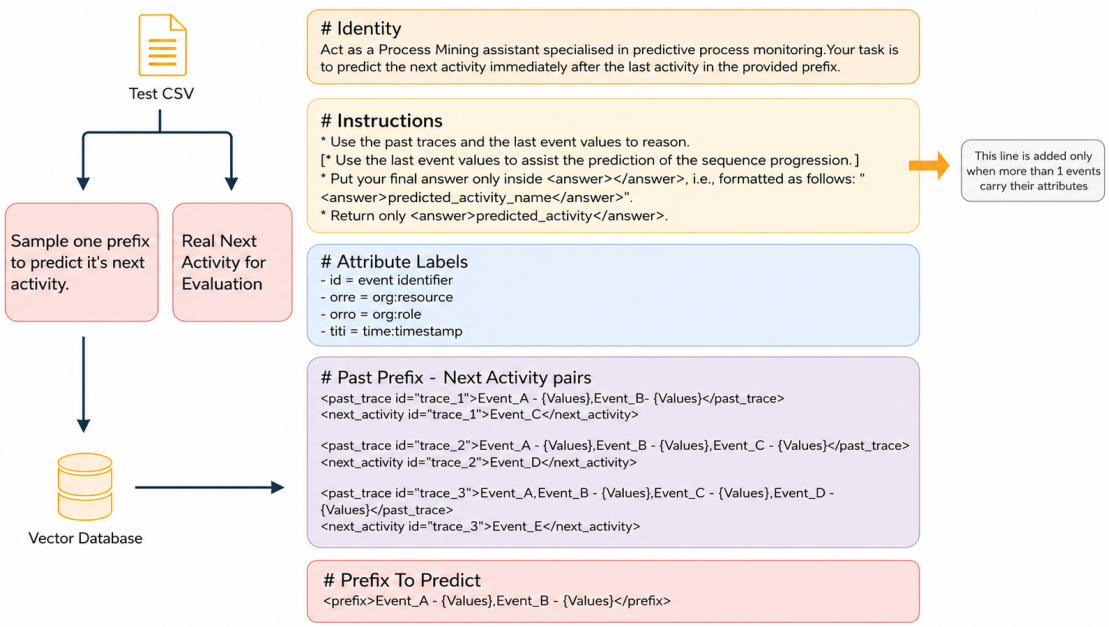

Figure 3.3: Vector database context within prompt construction

3.6 Prediction Flow

Once the final prompt has been assembled, it is submitted to the selected language model in order to generate the next-activity prediction for the query prefix. At this stage, the prompt already

contains the task instructions, the retrieved historical examples, the attribute legend when available, and the prefix to be predicted. The role of the model is therefore not to reconstruct the context, but to infer the most plausible activity that should follow the current execution state.

To make the output interpretable and machine-readable, the model is asked to return its final answer in a strictly constrained form, namely inside `<answer></answer>` tags. In this way, the prediction stage does not depend on free-form generated text, but on an explicitly delimited output segment that can be isolated reliably even when the model produces additional reasoning or explanatory material. The generation itself is performed in a deterministic setting, so that the variability introduced by sampling is minimized during evaluation.

After the model response is received, the pipeline extracts the content enclosed within the `<answer></answer>` tags and treats this text as the predicted next activity. The extracted label is then normalized so that formatting differences do not affect the comparison with the ground truth. In particular, surrounding spaces and quotation marks are removed, and the label is converted to a standardized lowercase form before evaluation.

If the response does not contain a valid answer block, or if the extracted content corresponds to an empty or null-like output, the prediction is treated as invalid and is not counted as a successful evaluated case. Only responses that can be parsed into a valid activity label are retained as final predictions. For each accepted case, the normalized predicted label is stored together with the corresponding ground-truth next activity of the test row.

The final result of this stage is therefore a collection of prediction–target pairs, one for each valid query prefix processed during the evaluation run. These pairs form the direct input to the metric computation stage, where correctness is assessed and aggregate performance values such as accuracy, macro-precision, macro-recall, macro-F_1, and earliness-oriented bucketed metrics are calculated.

Chapter 4

Experiments

4.1 Experimental Setting

This section summarizes the main components and configuration choices that define the experimental environment. It first presents the event logs used in the study and then outlines the embedding models, reranking component, and large language models considered in the evaluation.

4.1.1 Datasets

The evaluation uses four publicly available real-life event logs that are widely used in predictive process monitoring and process mining research. Together, they cover financial, administrative, and healthcare processes, and they differ substantially in size, control-flow variability, and attribute richness. This diversity is useful for assessing whether the proposed retrieval-augmented approach remains effective across logs with different behavioral structure and operational context.

The *BPI Challenge 2012* log captures the handling of personal loan and overdraft applications at a Dutch financial institution. It contains 262,200 events distributed across 13,087 traces. The process includes application registration, automated and manual checks, offer creation, offer acceptance, and final completion or cancellation, making it a strong benchmark for next-activity prediction under branching behavior and decision-intensive flows. In the local XES version used in this thesis, the log spans from October 1, 2011 to March 14, 2012, and it contains 24 distinct activity labels, or 36 activity classes when lifecycle transitions are taken into account [36].

The *BPI Challenge 2020 International Declarations* log contains travel-declaration and reimbursement cases related to international trips. It comprises 6,449 cases and 72,151 events. The process includes permit submission, approval steps by different organizational roles, declaration handling, and payment-related follow-up actions. Compared with the 2012 loan-application log, this dataset is smaller but still behaviorally rich, with repeated approval patterns, role-dependent routing, and multiple administrative checkpoints. In the local XES file used here, the observed event timestamps range from October 5, 2016 to May 10, 2020, and the log contains 34 distinct activity labels [37].

The *Sepsis Cases* event log captures the trajectories of patients treated for sepsis in a hospital environment. It contains 1,050 traces and 15,214 events covering 16 distinct activities, along with a relatively rich set of recorded attributes, including organizational groups, laboratory-test results, and checklist-related information. Because the cases reflect real clinical pathways, the log is less regular than strongly structured administrative datasets and offers a useful benchmark for testing how the proposed method behaves under noisy and medically heterogeneous process executions. This intuition is also supported by recent evidence showing that the Sepsis Cases log exhibits increased complexity and variability, especially for longer prefixes, which makes next-activity prediction more challenging [22]. In the local XES file used in this thesis, the timestamps range from November 7, 2013 to June 5, 2015 [38].

The *Hospital Billing* event log was extracted from the financial modules of a regional hospital ERP system. It contains 100,000 traces and 451,359 events, recording the activities required to bill bundled medical-service packages. The log does not describe the clinical treatment itself; instead, it focuses on the administrative billing workflow. This makes it particularly useful

for evaluating predictive methods in large-scale, repetitive back-office processes with moderate control-flow variation and substantial volume. In the present study, this dataset was not part of the baseline evaluation and was used mostly in the split-method evaluation. The local XES version spans from December 13, 2012 to January 19, 2016 and contains 18 distinct activity labels [39].

Table 4.1: Summary of the real-world event logs used in the evaluation

Dataset	Time Span (months)	Traces	Events	Activity Labels
BPI Challenge 2012	5.4	13,087	262,200	24
BPI Challenge 2020 International Declarations	43.1	6,449	72,151	34
Sepsis Cases	18.9	1,050	15,214	16
Hospital Billing	37.2	100,000	451,359	18

4.1.2 Embedding Models

In the retrieval component, two embedding models were examined. A MiniLM-based sentence-transformer model was selected as a reference point because it was also adopted in prior RAG-based work on next-activity prediction and yielded strong results there, making it an appropriate baseline for comparison [22]. In parallel, a more recent and explicitly retrieval-oriented BGE model was evaluated in order to assess whether a newer embedding family designed more directly for dense retrieval could offer additional benefits. This comparison made it possible to contrast a strong prior reference encoder with a more recent retrieval-focused alternative under a similar retrieval pipeline.

The MiniLM configuration uses `sentence-transformers/all-MiniLM-L12-v2` [40]. According to its model card, it maps sentences and short paragraphs to a 384-dimensional dense vector space and is intended for semantic search, clustering, and similarity-based retrieval, which makes it a reasonable lightweight baseline for prefix retrieval in process monitoring. Its underlying model family is rooted in MiniLM, which was designed through deep self-attention distillation to retain strong transformer behavior in a much smaller architecture [26]. In this thesis, MiniLM serves primarily as a compact baseline for testing whether a lighter encoder can still retrieve useful precedent prefixes.

By contrast, the BGE setting relies on `BAAI/bge-small-en-v1.5` [41]. This model belongs to the BGE family, which is motivated by recent work on embedding models optimized for retrieval-oriented use cases, including strong multilinguality, multiple retrieval functions, and improved granularity through self-knowledge distillation [24]. Although the exact deployed model in the code is the smaller English `v1.5` variant, with a 384-dimensional representation space and a maximum input length of 512 tokens according to its model card, it follows the same broader design philosophy of the BGE line and is therefore a suitable retrieval-focused choice for the dense nearest-neighbor stage. In practice, it was selected as the main embedding model because it better reflects current retrieval-oriented embedding design while remaining lightweight enough for repeated experimentation.

4.1.3 Reranker

The reranking stage is a small extension of the retrieval pipeline. After the embedding model retrieves an initial pool of candidate prefixes from the vector database, a second model can be applied to reorder those candidates before the final top-k examples are passed to the predictor. In the implementation used here, this stage relies on `BAAI/bge-reranker-base`, which is loaded as a sequence-classification cross-encoder and scores query–candidate pairs directly rather than comparing independent embeddings [42]. This design follows the standard reranking setting

introduced in neural passage reranking with BERT-style models [30].

The main motivation for including this component was to test whether a more precise second-stage ranking step could improve the quality of the retrieved context with only a limited extension of the overall method. Since the reranker is applied only to a small candidate pool returned by dense retrieval, it preserves the general structure of the approach while offering a more targeted relevance assessment before prompt construction. In this way, reranking functions as a lightweight refinement of retrieval.

4.1.4 Large Language Models

The evaluation considered a mix of open-weight local models served through Ollama and proprietary models accessed through the OpenAI API. The goal was not to provide an exhaustive architectural analysis, but rather to compare models of different scale, design, and deployment setting under the same prediction task. Table 4.2 summarizes the main LLMs used in the experiments together with the model information reported by their corresponding providers [43–48].

Table 4.2: Summary of the LLMs considered in the evaluation

Model	Size	Architecture	Context	Embedding	Quant.
`gemma3:4b`	4.3B	gemma3	131,072	2,560	Q4_K_M
`phi4:14b`	14.7B	phi3	16,384	5,120	Q4_K_M
`mistral-small3.2:24b`	24.0B	mistral3	131,072	5,120	Q4_K_M
`qwen3:30b`	30.5B	qwen3moe	262,144	2,048	Q4_K_M
`gpt-4o-mini`	N/P	Proprietary OpenAI	128,000	N/P	N/P
`gpt-4.1-mini-2025-04-14`	N/P	Proprietary OpenAI	1,047,576	N/P	N/P

The open-weight LLMs were served locally through Ollama (v0.30.10) on a workstation running Ubuntu 22.04.5 LTS. The platform featured an AMD Ryzen Threadripper PRO 5975WX processor with 32 cores, 251 GiB of system memory, and two NVIDIA GeForce RTX 4090 GPUs. Each GPU provided 24,564 MiB of VRAM, corresponding to approximately 24 GiB per device and about 48 GiB in total. These resources enabled the local execution of the quantized Ollama models used in the evaluation.

4.1.5 Evaluation Metrics

The evaluation is based on multi-class next-activity prediction. Let N denote the number of evaluated prefixes, let y_i be the true next-activity label of the i-th prefix, and let $\hat{y}_i$ be the corresponding predicted label. The set of activity classes is denoted by $\mathcal{C}$.

Accuracy measures the proportion of correct predictions over all evaluated cases:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i),$$

where $\mathbf{1}(\cdot)$ is the indicator function. This metric captures overall correctness, but it can be dominated by frequent activity classes when the label distribution is imbalanced.

For each class $c \in \mathcal{C}$, class-specific precision and recall are computed from the corresponding true positives (TP_c), false positives (FP_c), and false negatives (FN_c):

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}.$$

Precision indicates how reliable the predictions of class c are, whereas recall expresses how well the model recovers the true occurrences of that class.

To avoid overemphasizing dominant classes, macro-averaged precision and recall are used:

$$\text{Precision}_{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{Precision}_c, \qquad \text{Recall}_{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{Recall}_c.$$

These averages assign equal weight to each activity class, making them especially informative when the next-activity distribution is uneven across the log.

For each class, the F_1 score is defined as the harmonic mean of precision and recall:

$$F_{1,c} = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

The reported macro-F_1 is then

$$F_{1,\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F_{1,c}.$$

Macro-F_1 is useful because it balances false positives and false negatives while still treating all activity classes equally.

In addition to these aggregate classification metrics, the evaluation also includes earliness-oriented analysis based on prefix-length buckets [22]. Let ℓ_i denote the length of the i-th evaluated prefix, and let

$$B_1, B_2, \ldots, B_m$$

be the prefix-length intervals induced by the selected earliness boundaries. Each evaluated case is assigned to exactly one bucket according to its prefix length. For a bucket B_j, the share of evaluated prefixes that fall into that interval is

$$\text{Share}(B_j) = \frac{|B_j|}{N},$$

where $|B_j|$ is the number of evaluated prefixes in bucket B_j. This quantity indicates how much of the evaluation corresponds to earlier or later stages of process execution.

The same classification metrics are then recomputed separately inside each bucket. If B_j contains N_j evaluated prefixes, its bucket-wise accuracy is

$$\text{Accuracy}(B_j) = \frac{1}{N_j} \sum_{i \in B_j} \mathbf{1}(\hat{y}_i = y_i),$$

and analogously bucket-wise macro-precision, macro-recall, and macro-F_1 are computed by restricting the evaluation to the subset of prefixes in B_j. In this way, the earliness analysis does not introduce a different notion of correctness; rather, it shows how predictive performance changes as more or less of the process has already been observed.

Taken together, the aggregate and earliness-oriented metrics provide a broader view of predictive quality. In addition, the specifications of the pipeline components themselves, such as the embedding model, reranking setting, prompt variant, and LLM configuration, are also used as comparison dimensions in the experimental analysis.

4.2 Results

This section presents and discusses the evaluation results obtained from the pipeline configuration described above. The analysis focuses on how the proposed approach behaves under different settings across the examined event logs.

4.2.1 RQ1: How does the choice of large language model affect prediction quality in the proposed RAG-based next-activity prediction framework?

To address RQ1, the evaluation compares six large language models within the same RAG-based next-activity prediction pipeline across the four event logs considered in this study. The goal of this comparison is to isolate the effect of the LLM choice itself while keeping the remaining components of the pipeline as stable as possible. In this way, differences in predictive performance can be interpreted primarily in relation to the reasoning and generation behavior of the examined models rather than to changes in retrieval or encoding design.

For this reason, a fixed baseline configuration was adopted for the rest of the pipeline. Prefix generation was performed with $g = 3$, which produces a leaner but still representative set of prefixes for the comparison. In addition, the value-retention parameter was set to $m = 1$, meaning that the full activity sequence remained visible in each prefix while detailed attribute values were preserved only for the most recent event. This provided a simple and controlled prompt setting for the LLM comparison.

On the retrieval side, `bge-small-en-v1.5` was used as the embedding model, since it served as the main retrieval-oriented baseline of the study. No reranker was applied in this part of the analysis, so that the observed differences would not be confounded by a second-stage ranking component. The influence of prefix construction choices, embedding models, and reranking is examined separately in RQ2 and RQ3; therefore, RQ1 focuses specifically on how the final predictive quality changes when only the LLM is varied.

The obtained results are discussed below on a per-log basis.

Table 4.3: LLM comparison on BPI Challenge 2012 under the fixed RQ1 configuration

LLM	Acc.	Prec. (macro)	Rec. (macro)	F1 (macro)
Gemma 3 4B	0.4733	0.3877	0.3482	0.3419
Phi-4 14B	0.5300	0.4698	0.3684	0.3898
Mistral Small 3.2 24B	0.6200	0.5952	0.5216	0.5395
Qwen3 30B	0.6033	0.5171	0.4943	0.4916
GPT-4o mini	0.5600	0.4630	0.4373	0.4337
GPT-4.1 mini	0.6600	0.6767	0.5440	0.5787

Several patterns emerge from these results. On *BPI Challenge 2012*, *GPT-4.1 mini* achieved the strongest overall performance, reaching an accuracy of 0.6600 and the highest macro-F_1 score of 0.5787. *Mistral Small 3.2 24B* followed closely and showed a strong balance across all four metrics, while *Gemma 3 4B* remained the weakest model on this log. This indicates that, for a behaviorally richer and more branching process such as BPI 2012, more capable models tend to better exploit the retrieved contextual examples.

Table 4.4: LLM comparison on BPI Challenge 2020 International Declarations under the fixed RQ1 configuration

LLM	Acc.	Prec. (macro)	Rec. (macro)	F1 (macro)
Gemma 3 4B	0.6000	0.3530	0.3235	0.3211
Phi-4 14B	0.7067	0.4124	0.3946	0.3837
Mistral Small 3.2 24B	0.7800	0.5069	0.4103	0.4279
Qwen3 30B	0.8167	0.5613	0.5893	0.5648
GPT-4o mini	0.7500	0.5406	0.5122	0.5134
GPT-4.1 mini	0.8033	0.5552	0.5605	0.5490

On *BPI Challenge 2020 International Declarations*, the overall pattern became even clearer. *Qwen3 30B* obtained the best results on all reported metrics, with an accuracy of 0.8167,

macro-recall of 0.5893, and macro-F_1 of 0.5648. *GPT-4.1 mini* was very close in accuracy at 0.8033, while *Mistral Small 3.2 24B* also remained competitive at 0.7800 accuracy. In contrast, *Gemma 3 4B* and *Phi-4 14B* trailed more clearly behind. The gap between the stronger and weaker models is therefore most visible on this dataset, suggesting that LLM choice has substantial impact when the process contains repeated but still non-trivial administrative routing patterns.

Table 4.5: LLM comparison on Sepsis Cases under the fixed RQ1 configuration

LLM	Acc.	Prec. (macro)	Rec. (macro)	F1 (macro)
Gemma 3 4B	0.4733	0.3455	0.3180	0.3249
Phi-4 14B	0.4867	0.4358	0.3551	0.3721
Mistral Small 3.2 24B	0.5000	0.4104	0.3608	0.3657
Qwen3 30B	0.4800	0.4298	0.3933	0.4016
GPT-4o mini	0.4867	0.4401	0.4045	0.4114
GPT-4.1 mini	0.4967	0.3960	0.3277	0.3427

The *Sepsis Cases* log produced a more compressed ranking. Here, all models performed more modestly, which is consistent with the greater irregularity and heterogeneity of this healthcare dataset. *Mistral Small 3.2 24B* achieved the highest accuracy at 0.5000, but *GPT-4o mini* obtained the best macro-precision, macro-recall, and macro-F_1 values. This is an important result, because it shows that the model with the highest accuracy is not necessarily the one with the best class-balanced behavior. In other words, on a more difficult and less regular log, the relative ranking depends more strongly on which evaluation metric is emphasized.

Table 4.6: LLM comparison on Hospital Billing under the fixed RQ1 configuration

LLM	Acc.	Prec. (macro)	Rec. (macro)	F1 (macro)
Gemma 3 4B	0.6300	0.4138	0.3953	0.3877
Phi-4 14B	0.6367	0.4489	0.4053	0.4123
Mistral Small 3.2 24B	0.6767	0.4993	0.4292	0.4307
Qwen3 30B	0.7167	0.4915	0.4810	0.4793
GPT-4o mini	0.6667	0.4592	0.4560	0.4552
GPT-4.1 mini	0.6600	0.4101	0.4223	0.4079

On *Hospital Billing*, *Qwen3 30B* again emerged as the strongest model overall, achieving the best values across accuracy, recall, and macro-F_1, while *Mistral Small 3.2 24B* ranked second in accuracy and macro-precision. *GPT-4o mini* also remained competitive, whereas *GPT-4.1 mini* did not preserve the same advantage it showed on BPI 2012. This indicates that superior overall performance is not attached to a single model in every log, but rather depends on how well the model adapts to the behavioral structure and class distribution of the specific dataset.

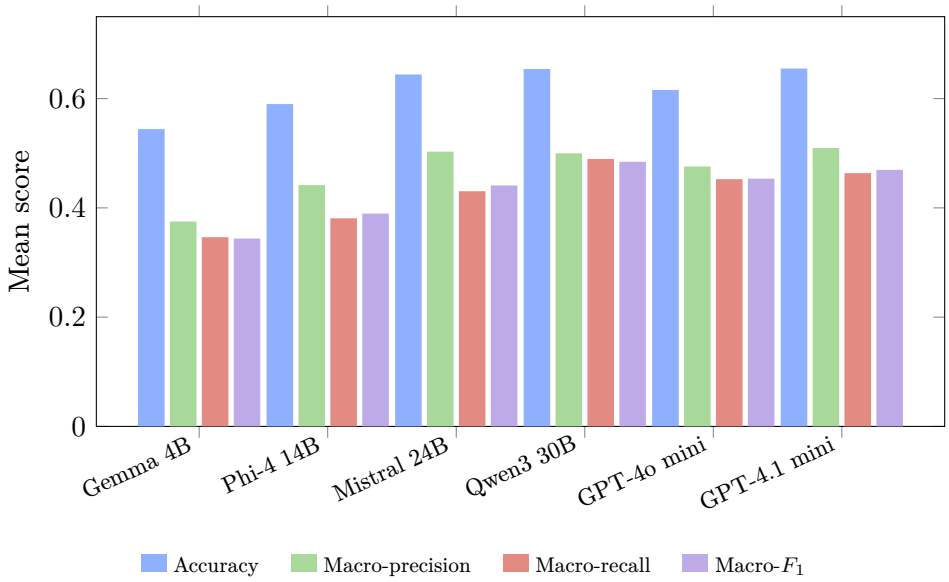

Figure 4.1: Mean metric comparison of the examined LLMs across the four event logs under the fixed RQ1 configuration

Looking across all four logs together, two models stand out most consistently: *GPT-4.1 mini* and *Qwen3 30B*. Their mean accuracies across the four datasets were 0.6550 and 0.6542, respectively, making them almost indistinguishable in overall accuracy. However, *Qwen3 30B* achieved slightly stronger mean macro-recall and macro-F_1, indicating somewhat better balanced performance across activity classes, whereas *GPT-4.1 mini* retained a small advantage in mean macro-precision. *Mistral Small 3.2 24B*, with mean accuracy 0.6442, formed the strongest local baseline and remained competitive across all logs, which supports its use as a practical baseline model in the rest of the study. At the opposite end, *Gemma 3 4B* was consistently the weakest model under this configuration. In broad terms, this pattern is compatible with the expectation that models with greater capacity often exhibit stronger reasoning and decision behavior. At the same time, the effect is clearly not proportional to parameter count alone: although *Qwen3 30B* has about seven times as many parameters as *Gemma 3 4B*, its mean improvement across the four reported metrics is about 32%, which is substantial but far from a linear scaling effect.

Overall, the results show that LLM choice has a clear effect on prediction quality in the proposed RAG-based next-activity prediction framework. The magnitude of that effect is dataset-dependent: it is more pronounced on the BPI 2012, BPI 2020, and Hospital Billing logs, while it becomes narrower on Sepsis Cases, where all models face a more difficult prediction setting. The comparison also shows that larger or stronger-performing models do not improve the results uniformly across all metrics, but they do tend to offer a more reliable upper range of performance. For this reason, parameter count is useful here as an interpretive signal, but not as a sufficient explanation on its own; architecture, instruction tuning, and retrieval-grounded response behavior also appear to play an important role. This confirms that the LLM is a meaningful design choice in the pipeline rather than a neutral interchangeable component.

Mistral Small 3.2 24B offers the strongest trade-off among the local models, combining competitive predictive performance with a more practical deployment profile. For this reason, it is used as the baseline LLM in the subsequent experiments.

4.2.2 RQ2: How do prefix-construction choices, particularly the gap and last-m parameters, influence the predictive performance of the proposed framework?

This subsection examines how prefix-construction choices shape the behavior of the proposed framework under a fixed retrieval and generation setup. In this way, the analysis attempts to assess not only whether different prefix encodings change predictive performance, but also how they alter the balance between control-flow coverage and the amount of recent attribute information retained in the textual representation. To keep the comparison focused, the experiments in this part use *Mistral Small 3.2 24B* as the baseline LLM and `bge-small-en-v1.5` as the embedding model.

For the effect of the gap parameter, the analysis relies on four configurations: a gap of 1 with prefixes encoded using the attributes of only the last observed event ($g = 1, m = 1$), a gap of 1 with prefixes encoded using the attributes of the last three events ($g = 1, m = 3$), a gap of 3 with prefixes encoded using the attributes of only the last observed event ($g = 3, m = 1$), and a gap of 3 with prefixes encoded using the attributes of the last three events ($g = 3, m = 3$). This comparison makes it possible to contrast denser prefix sampling against a more selective construction strategy while controlling for the retained value context. In practical terms, it allows the results to show whether exposing the model to more closely spaced historical prefixes leads to better predictions, or whether a leaner prefix set is sufficient when retrieval and generation remain unchanged.

Table 4.7: Effect of the gap parameter under the last-event representation ($m = 1$)

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	Gap 1	Gap 3	Gap 1	Gap 3	Gap 1	Gap 3	Gap 1	Gap 3
BPI Challenge 2012	0.6767	0.6200	0.5450	0.5952	0.5417	0.5216	0.5376	0.5395
BPI Challenge 2020 International Declarations	0.8400	0.7800	0.6123	0.5069	0.5928	0.4103	0.5907	0.4279
Sepsis Cases	0.4900	0.5000	0.3677	0.4104	0.3633	0.3608	0.3364	0.3657
Hospital Billing	0.7467	0.6767	0.5837	0.4993	0.5455	0.4292	0.5534	0.4307

The results reveal a generally consistent pattern, while also highlighting a few dataset-specific deviations. Under the last-event representation ($m = 1$), moving from gap 3 to gap 1 improves accuracy by $+0.0567$ on *BPI Challenge 2012*, $+0.0600$ on *BPI Challenge 2020 International Declarations*, and $+0.0700$ on *Hospital Billing*. The corresponding macro-F_1 differences are almost neutral on BPI 2012 (-0.0020), but clearly positive on BPI 2020 ($+0.1628$) and Hospital Billing ($+0.1227$). Only *Sepsis Cases* deviates from this pattern, where gap 3 yields a small advantage in accuracy and macro-F_1, corresponding to -0.0100 and -0.0293 when the comparison is expressed as $(g = 1) - (g = 3)$. This already suggests that the denser configuration is usually beneficial, but not universally so.

Table 4.8: Effect of the gap parameter under the last-three-events representation ($m = 3$)

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	Gap 1	Gap 3	Gap 1	Gap 3	Gap 1	Gap 3	Gap 1	Gap 3
BPI Challenge 2012	0.6200	0.6000	0.4762	0.4943	0.4527	0.4625	0.4530	0.4630
BPI Challenge 2020 International Declarations	0.7967	0.7133	0.5805	0.3947	0.5507	0.3869	0.5453	0.3758
Sepsis Cases	0.4967	0.5400	0.3472	0.4327	0.3543	0.4020	0.3252	0.3893
Hospital Billing	0.6900	0.5700	0.5331	0.3742	0.4789	0.3772	0.4765	0.3661

The same comparison becomes even more informative under the last-three-events representation ($m = 3$). Here, gap 1 remains clearly stronger on *BPI Challenge 2020 International Declarations* and *Hospital Billing*, with macro-F_1 gains of $+0.1695$ and $+0.1103$, respectively, and accuracy gains of $+0.0833$ and $+0.1200$. By contrast, *Sepsis Cases* now shows a more visible reversal, with gap 3 outperforming gap 1 by 0.0433 in accuracy and 0.0641 in macro-F_1. *BPI Challenge 2012* again remains comparatively stable: its accuracy changes only by $+0.0200$, while macro-F_1 changes by -0.0100. This makes BPI 2012 the least sensitive of the four logs to the gap manipulation, whereas BPI 2020 and Hospital Billing are the most clearly affected.

These comparison patterns are summarized visually in Figure 4.2, which presents the metric differences between the gap-1 and gap-3 settings for both prefix representations.

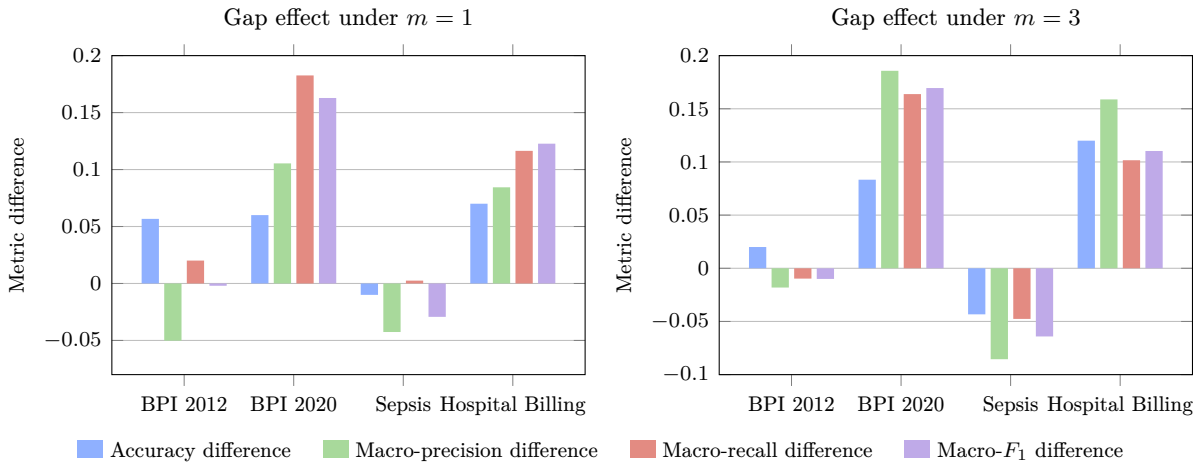

Figure 4.2: Metric differences for the gap comparison under the last-event representation ($m = 1$) and the last-three-events representation ($m = 3$), computed as (gap 1) − (gap 3). Positive values favor gap 1, while negative values favor gap 3.

An additional observation is that the effect does not appear to be explained simply by raw log size, average trace length, or the number of distinct activity labels. For example, BPI 2020 contains the largest activity-label set among the four logs, whereas Hospital Billing contains a much smaller one, yet both benefit strongly from the denser gap-1 setting. Likewise, Sepsis and Hospital Billing have relatively similar activity-label cardinalities, but they react quite differently to the same gap change. This suggests that the decisive factor is not merely how many activity labels exist in the log, nor how long traces are on average, but whether the removed intermediate prefixes carry retrieval evidence that is genuinely useful for the running case. Overall, the results support gap 1 as the safer default choice for this framework, while also showing that larger gaps can still be advantageous in logs where denser prefix generation appears to preserve more redundancy or noise than useful additional context.

For the effect of the *last-m* encoding itself, the comparison fixes the gap at 1 and evaluates prefixes encoded using the attributes of the last observed event ($m = 1$), the last three events ($m = 3$), and the last five events ($m = 5$). This part of the analysis isolates how much recent attribute history should be preserved explicitly in the prefix representation, while keeping the retrieval density fixed. As a result, it provides a more direct view of whether extending the visible recent attribute context helps the model reason more effectively about the next activity, or whether a shorter retained window is already sufficient.

In order to present this comparison in a compact but still informative way, the results are summarized through accuracy and macro-F_1. Accuracy is retained because it provides the clearest overall view of top-level predictive correctness, while macro-F_1 is included because it reflects class-balanced behavior and is therefore more sensitive to whether richer attribute retention helps across the full activity-label space rather than only on dominant classes. In addition, the analysis is extended beyond the baseline *Mistral Small 3.2 24B* model to include *GPT-4o mini* and *GPT-4.1 mini*, since the usefulness of retaining more recent values appears to depend not only on the prefix representation itself but also on the capability of the downstream LLM receiving that representation.

Table 4.9: Accuracy under the *last-m* comparison across the examined LLMs

Dataset	Mistral Small 3.2 24B			GPT-4o mini			GPT-4.1 mini		
	m 1	m 3	m 5	m 1	m 3	m 5	m 1	m 3	m 5
BPI Challenge 2012	0.6767	0.6200	0.6100	0.6067	0.6033	0.6600	0.6267	0.6933	0.6267
BPI Challenge 2020 International Declarations	0.8400	0.7967	0.8000	0.8333	0.8233	0.8467	0.8400	0.8533	0.8733
Sepsis Cases	0.4900	0.4967	0.4767	0.4467	0.5100	0.4267	0.4933	0.5233	0.5300
Hospital Billing	0.7467	0.6900	0.5667	0.7100	0.7000	0.5733	0.7600	0.7300	0.6633

Table 4.9 already shows that the effect of retaining more recent values is not uniform across models or datasets. Under the baseline *Mistral Small 3.2 24B* setting, ($m = 1$) remains the strongest choice on *BPI Challenge 2012*, *BPI Challenge 2020 International Declarations*, and *Hospital Billing*, while *Sepsis Cases* shows only a small accuracy increase under ($m = 3$). *GPT-4o mini* produces a more mixed picture: ($m = 5$) is strongest on *BPI Challenge 2012* and *BPI Challenge 2020 International Declarations*, ($m = 3$) is best on *Sepsis Cases*, and ($m = 1$) remains slightly preferable on *Hospital Billing*. By contrast, *GPT-4.1 mini* benefits more clearly from richer recent-value retention on three of the four datasets, with ($m = 3$) best on *BPI Challenge 2012* and ($m = 5$) best on both *BPI Challenge 2020 International Declarations* and *Sepsis Cases*; only *Hospital Billing* continues to favor ($m = 1$). At the level of overall accuracy, then, the retained-value effect is not governed by a single monotonic pattern, but stronger models appear more capable of exploiting the larger textual prefixes created by higher m values.

Table 4.10: Macro-F_1 under the *last-m* comparison across the examined LLMs

Dataset	Mistral Small 3.2 24B			GPT-4o mini			GPT-4.1 mini		
	m 1	m 3	m 5	m 1	m 3	m 5	m 1	m 3	m 5
BPI Challenge 2012	0.5376	0.4530	0.4572	0.4369	0.4098	0.5559	0.4667	0.5073	0.4736
BPI Challenge 2020 International Declarations	0.5907	0.5453	0.5620	0.6611	0.5895	0.6420	0.5918	0.6393	0.7400
Sepsis Cases	0.3364	0.3252	0.3227	0.3197	0.3209	0.2662	0.2716	0.3096	0.3685
Hospital Billing	0.5534	0.4765	0.4119	0.5356	0.5438	0.5213	0.5697	0.5240	0.4585

The macro-F_1 comparison in Table 4.10 makes this interaction more explicit and is particularly informative because it reflects class-balanced performance rather than only aggregate correctness. For *Mistral Small 3.2 24B*, the pattern is consistently conservative: $(m = 1)$ is best on all four datasets, and increasing the retained recent-value window either reduces macro-F_1 directly or, at best, only recovers a small part of the loss at $(m = 5)$. *GPT-4o mini* remains more variable. It benefits strongly from $(m = 5)$ on *BPI Challenge 2012*, stays strongest at $(m = 1)$ on *BPI Challenge 2020 International Declarations*, changes only marginally on *Sepsis Cases*, and reaches its local maximum at $(m = 3)$ on *Hospital Billing*. The clearest contrast is provided by *GPT-4.1 mini*: on *BPI Challenge 2012*, *BPI Challenge 2020 International Declarations*, and *Sepsis Cases*, macro-F_1 improves when more recent values are retained, and on *BPI Challenge 2020 International Declarations* the gain from $(m = 1)$ to $(m = 5)$ is especially pronounced. Only *Hospital Billing* continues to deteriorate as m grows.

Several structural observations emerge from this cross-dataset view. First, the number of activity labels alone does not explain the pattern: *BPI Challenge 2020 International Declarations*, with the largest label set among the four logs, is precisely the dataset on which the stronger models benefit most clearly from richer value retention, whereas *Hospital Billing*, with only 18 labels, reacts negatively to larger m values for all three LLMs. Second, average trace length is not sufficient on its own either. Using the event and trace counts reported earlier, the logs differ substantially in mean trace length, with approximately 20 events per case in *BPI Challenge 2012*, 14.5 in *Sepsis Cases*, 11.2 in *BPI Challenge 2020 International Declarations*, and only about 4.5 in *Hospital Billing*. Yet the shortest log, *Hospital Billing*, is the one where additional retained value blocks seem least useful, which suggests that when traces are already short and operationally repetitive, extending the recent-value window may contribute more redundancy than discriminative evidence. By contrast, the more visible gains for *GPT-4.1 mini* on *BPI Challenge 2020 International Declarations* and *Sepsis Cases* indicate that richer value retention can become beneficial when the process contains more heterogeneous routing or attribute-rich context and the downstream LLM is capable of exploiting it. Taken together, these results suggest that retaining more recent values does not improve performance universally, but its usefulness increases when two conditions are jointly met: the additional value context is genuinely informative for the process, and the downstream LLM is strong enough to use the larger textual representation effectively.

Figure 4.3 makes the cross-model pattern easier to read at an aggregate level. Under the baseline *Mistral Small 3.2 24B* setting, all four mean-difference panels remain below zero for both $(m = 3)$ and $(m = 5)$, indicating that retaining more recent values leads to a consistent average decline across the examined datasets. *GPT-4o mini* appears more mixed: its mean changes remain negative in accuracy at $(m = 5)$, macro-precision at $(m = 3)$, and macro-F_1 at $(m = 3)$, but become slightly positive in macro-precision, macro-recall, and macro-F_1 at $(m = 5)$. By contrast, *GPT-4.1 mini* shows the most favorable aggregate profile, with positive mean differences in macro-precision, macro-recall, and macro-F_1 already at $(m = 3)$, and with the strongest positive shifts becoming visible at $(m = 5)$. At the same time, accuracy still does not improve monotonically for every model, which reinforces the broader point that richer value retention is more helpful for class-balanced behavior than for overall top-level correctness alone.

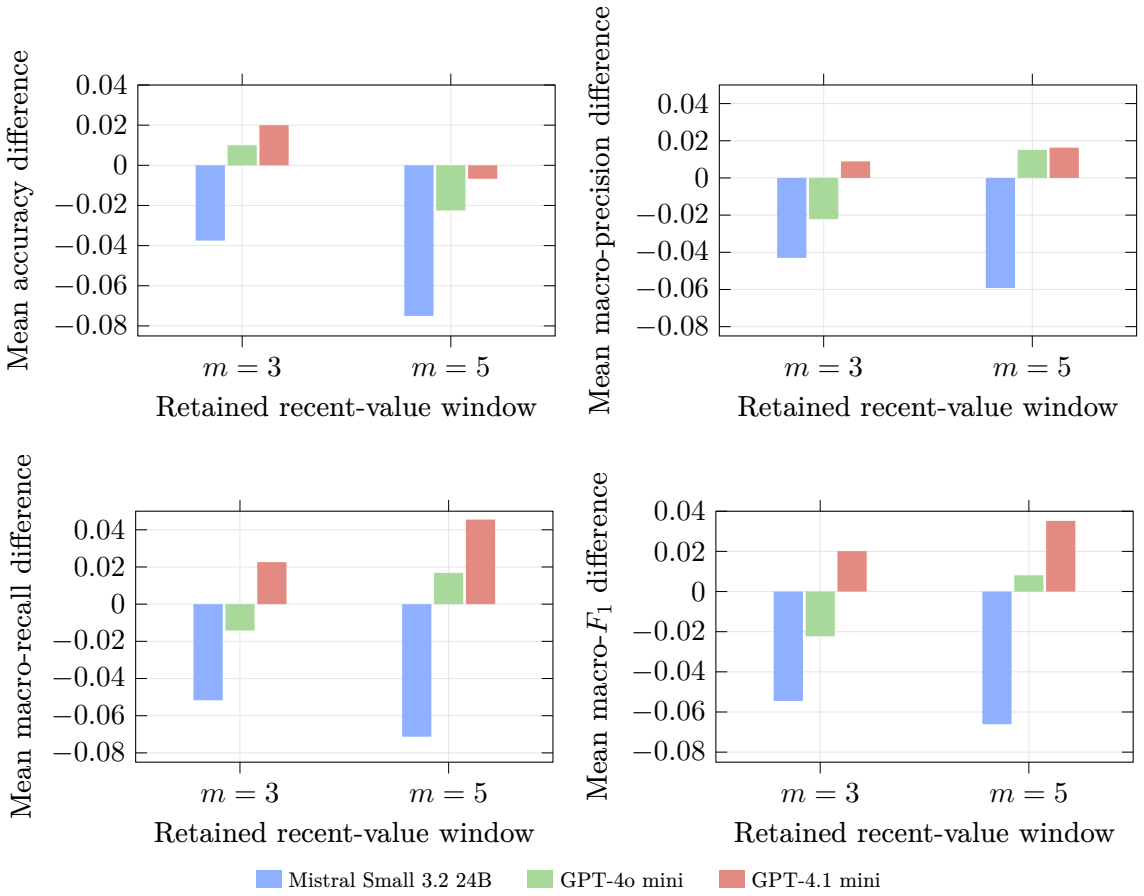

Figure 4.3: Mean metric differences across the four datasets under the *last-m* comparison. Each bar is computed relative to the $(m = 1)$ setting of the same LLM, that is, as $(m = 3) - (m = 1)$ or $(m = 5) - (m = 1)$. Values above zero indicate an average improvement over $(m = 1)$, while values below zero indicate an average decline.

4.2.3 RQ3: What indications do the evaluated runs provide about the effect of retrieval-side choices, including the embedding model and the optional reranking step, on final prediction performance?

This subsection examines retrieval-side choices under a fixed predictive setting in order to study how the embedding model and the retained recent-value context influence the behavior of the proposed framework. To keep the comparison focused, the downstream LLM is fixed to *Mistral Small 3.2 24B* as the baseline model, while the same gap setting $(g = 1)$ and the same retrieval budget of top-10 examples are used across all four datasets.

For the embedding-model comparison, the analysis relies on four aligned configurations: `all-MiniLM-L12-v2` with prefixes encoded using the attributes of only the last observed event $(m = 1)$, `all-MiniLM-L12-v2` with prefixes encoded using the attributes of the last three observed events $(m = 3)$, `bge-small-en-v1.5` with the same last-event representation $(m = 1)$, and `bge-small-en-v1.5` with the last-three-events representation $(m = 3)$. In this way, the subsection examines the same two prefix representations under both embedding models, so that the comparison remains centered on the retrieval side while the generation stage remains unchanged.

Table 4.11: BGE results under the embedding-model comparison setting

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	m 1	m 3	m 1	m 3	m 1	m 3	m 1	m 3
BPI Challenge 2012	0.6767	0.6200	0.5450	0.4762	0.5417	0.4527	0.5376	0.4530
BPI Challenge 2020 International Declarations	0.8400	0.7967	0.6123	0.5805	0.5928	0.5507	0.5907	0.5453
Sepsis Cases	0.4900	0.4967	0.3677	0.3472	0.3633	0.3543	0.3364	0.3252
Hospital Billing	0.7467	0.6900	0.5837	0.5331	0.5455	0.4789	0.5534	0.4765

Table 4.11 first provides the BGE side of the comparison under the same fixed setting used throughout this part of the analysis. In this sense, the $(m = 1)$ column restates the established baseline retrieval configuration, while the $(m = 3)$ column extends that reference by showing the corresponding results under the richer recent-value representation within the same evaluation frame. The overall pattern is consistent: the last-event representation remains stronger on *BPI Challenge 2012*, *BPI Challenge 2020 International Declarations*, and *Hospital Billing* across all four reported metrics, whereas *Sepsis Cases* shows only a very small accuracy increase under $(m = 3)$ while macro-precision, macro-recall, and macro-F_1 remain slightly higher under $(m = 1)$. The table therefore serves both to restate the BGE reference setting used elsewhere in the chapter and to make the metric-level effect of the alternative BGE encoding directly visible before the MiniLM comparison is considered.

Table 4.12: MiniLM results under the embedding-model comparison setting

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	m 1	m 3	m 1	m 3	m 1	m 3	m 1	m 3
BPI Challenge 2012	0.5900	0.5500	0.5176	0.4027	0.4899	0.4057	0.4847	0.3911
BPI Challenge 2020 International Declarations	0.7967	0.7900	0.5546	0.5421	0.5347	0.5368	0.5294	0.5321
Sepsis Cases	0.5300	0.4700	0.3735	0.2855	0.3584	0.2289	0.3374	0.2440
Hospital Billing	0.6967	0.6333	0.5313	0.5834	0.5108	0.5397	0.5125	0.5407

Table 4.12 shows a broadly similar directional effect of the prefix representation, but with a less stable overall profile than the BGE setting. Under MiniLM, the $(m = 1)$ representation remains stronger on *BPI Challenge 2012* and *Sepsis Cases* across all four metrics, while *BPI Challenge 2020 International Declarations* shows only a very small difference between $(m = 1)$ and $(m = 3)$. *Hospital Billing* is the main exception, where $(m = 3)$ yields stronger macro-precision, macro-recall, and macro-F_1, although accuracy remains higher under $(m = 1)$.

Compared with the BGE results, MiniLM is generally weaker on *BPI Challenge 2012* and *BPI Challenge 2020 International Declarations* for both prefix representations, whereas the relative picture becomes more mixed on *Sepsis Cases* and *Hospital Billing*. At the same time, the transition from $(m = 1)$ to $(m = 3)$, where the embedded prefixes become longer because they retain more recent attribute values, appears to be handled differently by the two encoders. Under BGE, this richer representation usually leads to moderate degradation but preserves a comparatively stable metric profile, with the ranking between datasets remaining broadly consistent. Under MiniLM, the same increase in prefix size is associated with stronger shifts across datasets and metrics, including clearer losses on *BPI Challenge 2012* and *Sepsis Cases*, near-neutral behavior on *BPI Challenge 2020 International Declarations*, and a partial macro-level improvement on *Hospital Billing*. Taken together, this suggests that BGE provides the more consistent retrieval reference in the present setting, while MiniLM remains competitive

in selected cases but appears more sensitive both to the particular dataset and to the expansion of the textual prefix representation.

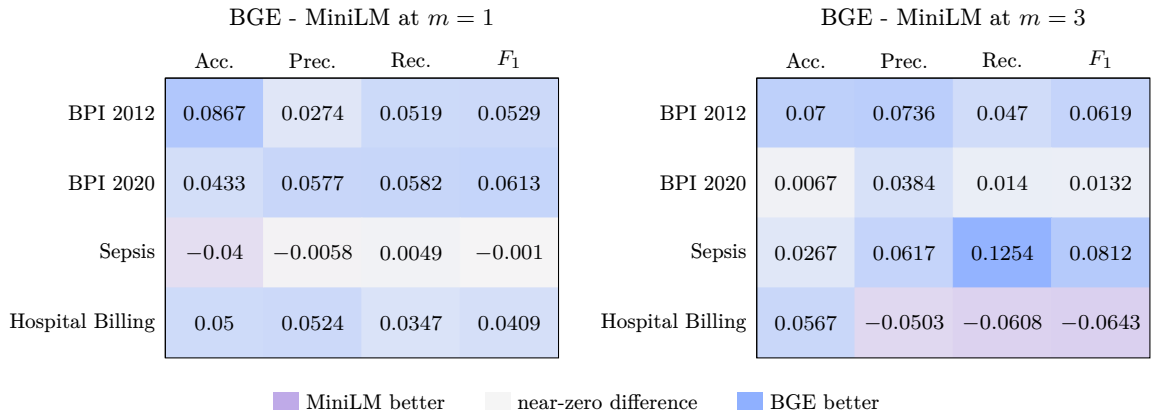

Figure 4.4: Metric differences between BGE and MiniLM under the baseline *Mistral Small 3.2 24B* setting, shown separately for the last-event representation ($m = 1$) and the last-three-events representation ($m = 3$). Positive values indicate stronger performance for BGE, while negative values indicate stronger performance for MiniLM.

The second part of RQ3 examines the optional reranking step separately, so that its effect is not conflated with the embedding-model choice. More specifically, the comparison contrasts the BGE retrieval setting without reranking against the corresponding reranked setting under the same gap configuration ($g = 1$) and the same last-event representation ($m = 1$), with reranking applied over a pool of 20 candidates before selecting the final top-10 examples. In this way, the analysis focuses on whether adding a second-stage ranking step changes the quality of the retrieved examples ultimately exposed to the baseline *Mistral Small 3.2 24B* model.

Table 4.13: Effect of reranking under the BGE retrieval setting

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	No reranker	BGE reranker	No reranker	BGE reranker	No reranker	BGE reranker	No reranker	BGE reranker
BPI Challenge 2012	0.6767	0.6667	0.5450	0.4996	0.5417	0.4729	0.5376	0.4785
BPI Challenge 2020 International Declarations	0.8400	0.8333	0.6123	0.6016	0.5928	0.5932	0.5907	0.5808
Sepsis Cases	0.4900	0.4667	0.3677	0.4216	0.3633	0.3881	0.3364	0.3876
Hospital Billing	0.7467	0.7433	0.5837	0.6592	0.5455	0.5899	0.5534	0.6046

The reranking comparison reveals a more mixed pattern than the embedding-model comparison. On *BPI Challenge 2012*, applying the BGE reranker reduces all four reported metrics, while on *BPI Challenge 2020 International Declarations* the effect is nearly neutral but still slightly negative overall, with only macro-recall remaining essentially unchanged. By contrast, *Sepsis Cases* and *Hospital Billing* show a different behavior: in both datasets, reranking leads to lower accuracy but improves the macro-level metrics, with the clearest gains appearing in macro-precision and macro-F_1. This indicates that the reranking stage does not consistently improve the dominant top-1 prediction behavior, but it seems more helpful when the dataset has class imbalance, heterogeneous behavior, or minority classes that are easier to overlook.

These metric-level observations become more apparent in the diverging bar visualization that follows, which makes the direction and relative magnitude of the reranking-induced changes easier to inspect across datasets and evaluation measures.

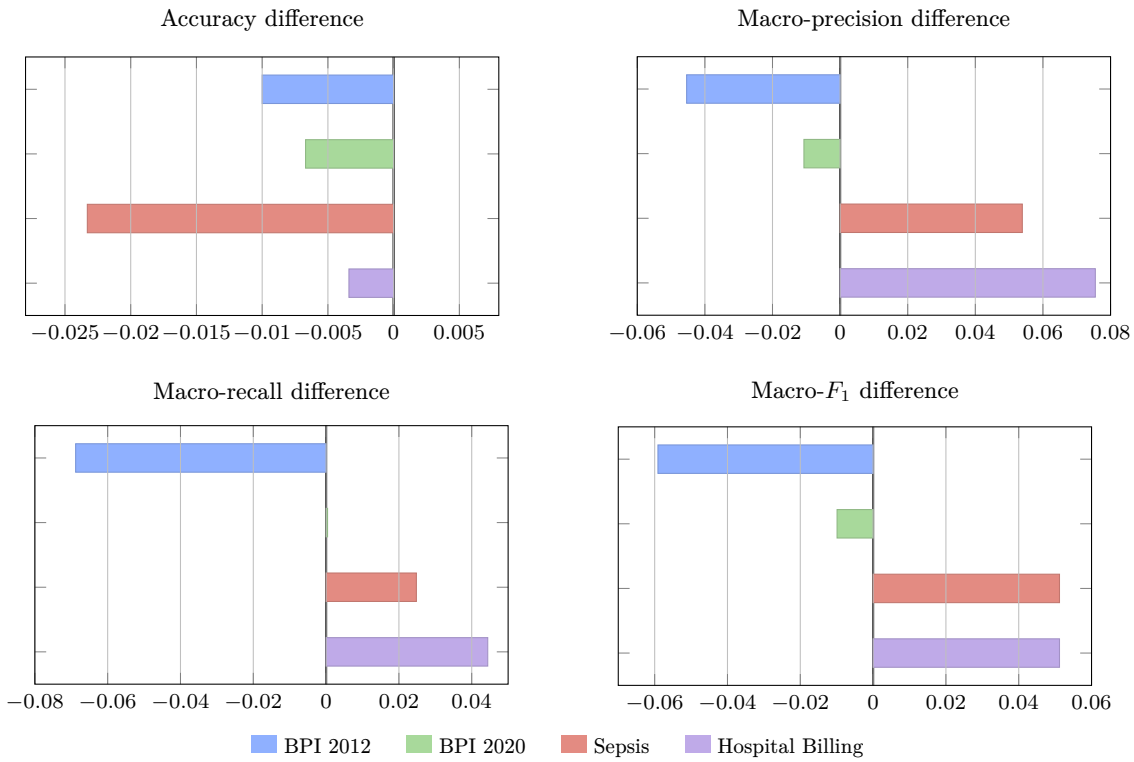

Figure 4.5: Diverging bar comparison of the reranking effect under the baseline *Mistral Small 3.2 24B* setting. Each value is plotted as (BGE reranker) − (no reranker), so positive values indicate improvement under reranking and negative values indicate deterioration.

4.2.4 RQ4: How does a more reliable and generalizable evaluation setting affect the prediction quality of the framework?

RQ4 shifts the focus from component-level comparisons to a broader evaluation of the overall pipeline as a next-activity prediction method. The aim is to verify whether the strongest configuration identified in the experiments remains effective under a more realistic evaluation setting, as described in Section 3.3.2, designed to strengthen the reliability and generalizability of the results. In this sense, the analysis is guided by issues such as the evaluation protocol itself and the robustness of the resulting performance under a stricter split strategy [3].

To keep this final comparison both focused and representative, the analysis centers on the *GPT-4.1 mini* setting under a gap of 1, while considering two retained recent-value configurations: prefixes encoded using the attributes of the last three observed events ($m = 3$) and prefixes encoded using the attributes of the last five observed events ($m = 5$). In practical terms, this means that the retrieval and prompting pipeline is examined under the denser prefix-sampling strategy that proved to be a strong-performing configuration earlier in the chapter, while the visible recent-value context is allowed to vary between a moderately rich and a more extensive representation. This choice is justified by the preceding results, where *GPT-4.1 mini* emerged as one of the most consistent and strongest-performing models overall, and where the ($g = 1$) setting combined with richer recent-value retention repeatedly produced competitive or leading results across several datasets. For this reason, the subsection uses these aligned ($g = 1, m = 3$) and ($g = 1, m = 5$) configurations as the basis for comparing the two split strategies under a common high-performing predictive setting.

Table 4.14: Comparison of prefix-level and trace-level split strategies under the *last-3*, *GPT-4.1 mini* configuration ($g = 1, m = 3$)

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	Prefix-level	Trace-level	Prefix-level	Trace-level	Prefix-level	Trace-level	Prefix-level	Trace-level
BPI Challenge 2012	0.6933	0.6600	0.5278	0.5805	0.5085	0.5360	0.5073	0.5343
BPI Challenge 2020 International Declarations	0.8533	0.8200	0.6352	0.5334	0.6557	0.5261	0.6393	0.5201
Sepsis Cases	0.5233	0.5300	0.3155	0.3250	0.3226	0.3316	0.3096	0.3268
Hospital Billing	0.7300	0.6467	0.5619	0.3728	0.5128	0.3692	0.5240	0.3597

Table 4.14 shows that the transition from the prefix-level split to the stricter trace-level split generally reduces predictive performance, although the extent of this reduction varies across datasets. In terms of accuracy, the trace-level split is lower on *BPI Challenge 2012*, *BPI Challenge 2020 International Declarations*, and *Hospital Billing*, with differences of -0.0333, -0.0333, and -0.0833, respectively, while *Sepsis Cases* is the only log that shows a small improvement of $+0.0067$. The macro-level metrics also indicate a dataset-dependent pattern. On *BPI Challenge 2012*, macro-precision, macro-recall, and macro-F_1 become slightly higher under the trace-level split despite the drop in accuracy, which suggests a modest gain in class-balanced behavior. By contrast, *BPI Challenge 2020 International Declarations* and especially *Hospital Billing* deteriorate across all reported metrics, with the strongest decline appearing on *Hospital Billing*, where macro-F_1 falls from 0.5240 to 0.3597. *Sepsis Cases* again behaves differently, since all four metrics increase slightly under the trace-level split, although the gains remain limited in absolute terms. Overall, the comparison indicates that the stricter split strategy tends to expose weaker generalization for this otherwise strong configuration, while also showing that the size of this effect depends substantially on the behavioral properties of the underlying event log.

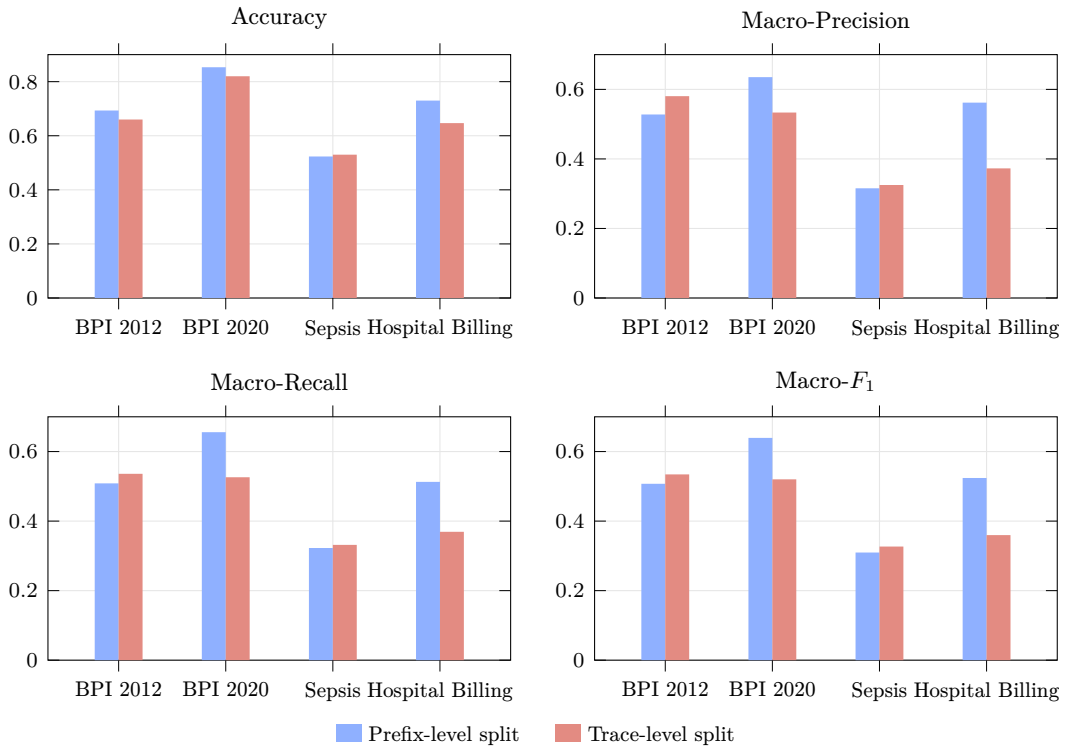

Figure 4.6: Metric-level comparison of the prefix-level and trace-level split strategies under the *last-3*, *GPT-4.1 mini* configuration ($g = 1, m = 3$).

At the same time, the absolute results remain sufficiently strong to suggest that the framework continues to behave reliably under this more demanding evaluation setting, while still remaining dependent on the particular log.

Table 4.15: Comparison of prefix-level and trace-level split strategies under the *last-5*, *GPT-4.1 mini* configuration ($g = 1, m = 5$)

Dataset	Accuracy		Macro-Precision		Macro-Recall		Macro-F_1	
	Prefix-level	Trace-level	Prefix-level	Trace-level	Prefix-level	Trace-level	Prefix-level	Trace-level
BPI Challenge 2012	0.6267	0.6400	0.4767	0.5158	0.4919	0.5157	0.4736	0.5026
BPI Challenge 2020 International Declarations	0.8733	0.8400	0.7410	0.6462	0.7491	0.6096	0.7400	0.6162
Sepsis Cases	0.5300	0.4700	0.3617	0.3126	0.3990	0.2834	0.3685	0.2872
Hospital Billing	0.6633	0.6267	0.4906	0.3221	0.4514	0.3113	0.4585	0.3095

Table 4.15 highlights a more polarized outcome under the *last-5* setting. Unlike the *last-3* comparison, where the stricter split still produced small gains on *Sepsis Cases*, the longer retained recent-value window now leads to a clearer separation between one favorable case and three unfavorable ones. *BPI Challenge 2012* is the only log in which the trace-level split improves all four reported metrics, raising accuracy from 0.6267 to 0.6400 and macro-F_1 from 0.4736 to 0.5026. This suggests that, for this dataset, the richer ($m = 5$) representation remains useful even when the evaluation is made stricter.

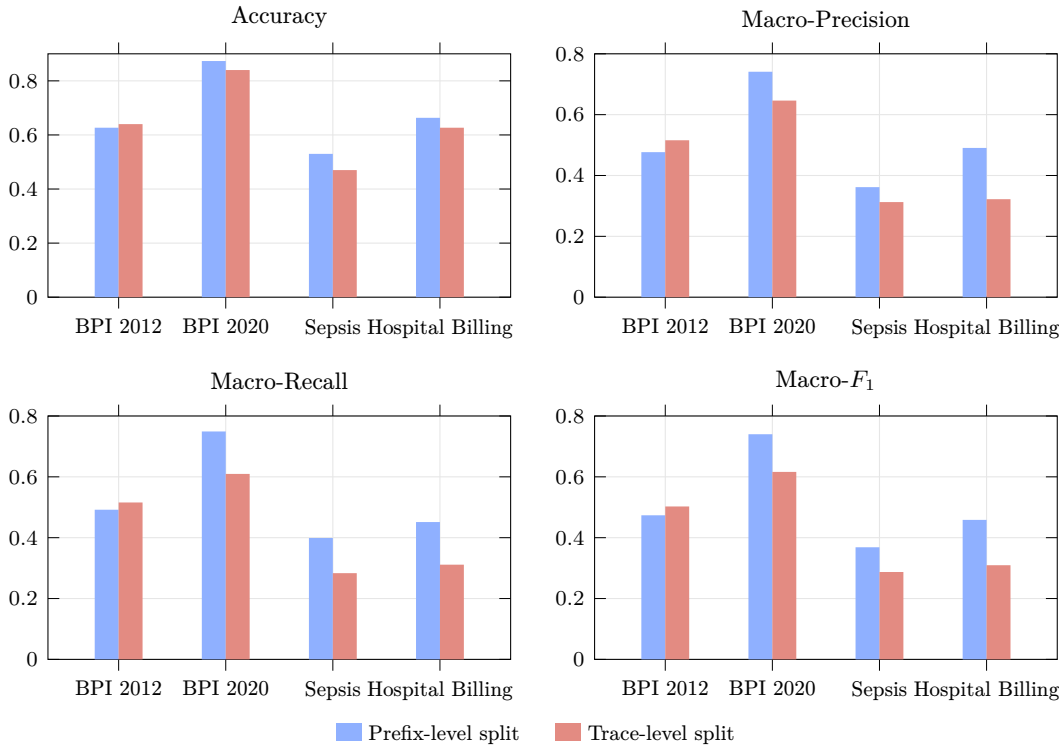

Figure 4.7: Metric-level comparison of the prefix-level and trace-level split strategies under the *last-5*, *GPT-4.1 mini* configuration ($g = 1, m = 5$).

The remaining three datasets follow the opposite direction. On *BPI Challenge 2020 International Declarations*, the drop in accuracy is moderate, but the reduction in class-balanced performance is much more pronounced, with macro-F_1 decreasing from 0.7400 to 0.6162. *Sepsis Cases* shows an even clearer decline across all metrics, indicating that the stronger prefix-level

result under ($m = 5$) does not transfer as well when overlap between training and evaluation traces is removed. The weakest outcome again appears on *Hospital Billing*, where the trace-level split substantially reduces macro-precision, macro-recall, and macro-F_1, confirming that this dataset remains especially sensitive to the stricter protocol. In this sense, Table 4.15 suggests that extending the retained recent-value context to five events can still be beneficial, but only in logs where that additional information remains genuinely discriminative under a more realistic split strategy.

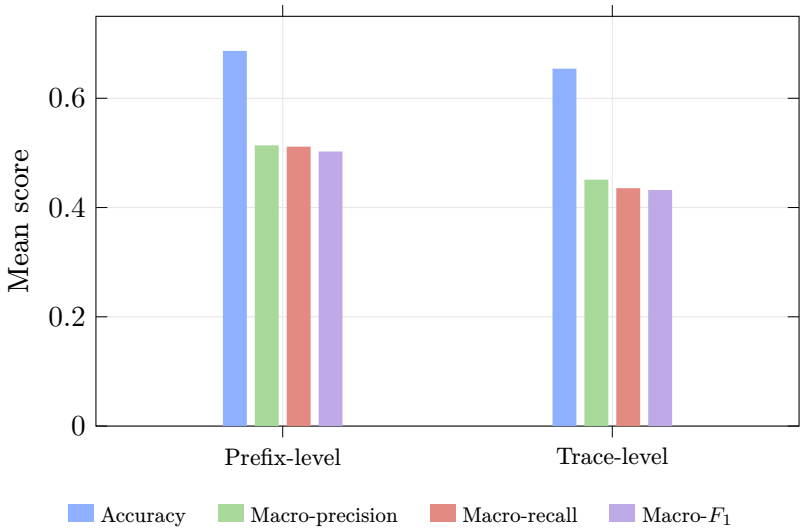

Figure 4.8: Mean metric comparison of the aggregated prefix-level and trace-level split results across the four datasets under the *GPT-4.1 mini* configurations with ($m = 3$) and ($m = 5$). Each bar represents the mean value of the corresponding metric over both retained recent-value settings and all four event logs.

Taken together, Figure 4.8 makes the aggregate comparison between the two split strategies more explicit. Across both retained recent-value settings and all four datasets, the prefix-level evaluation remains stronger on every reported metric, with mean accuracy decreasing from 0.6867 to 0.6542, mean macro-precision from 0.5138 to 0.4511, mean macro-recall from 0.5114 to 0.4354, and mean macro-F_1 from 0.5026 to 0.4321 when moving to the trace-level split. This confirms that the stricter evaluation protocol produces a measurable but not destructive reduction in observed performance. In this sense, the overall picture remains encouraging: even under the more realistic trace-level setting, the framework preserves a meaningful level of predictive effectiveness and therefore stands as a viable method for next-activity prediction, while its exact performance still depends on the characteristics of the underlying event log.

As an additional next-activity-prediction-focused evaluation measure, earliness expresses how soon the framework can produce reliable predictions while a case is still running. This is important in predictive process monitoring because a correct prediction is operationally more useful when it is available early enough to support an intervention [22]. Unlike aggregate accuracy, which summarizes all evaluated prefixes in a single value, the earliness view groups prefixes according to how many events have already been observed and then calculates next-activity prediction accuracy within each group. Performance in the shorter-prefix buckets therefore reflects the framework's ability to operate with limited case information, whereas performance in the longer-prefix buckets shows whether additional process context improves or destabilizes the prediction. Earliness should not be interpreted through prefix length alone: the number of evaluated prefixes in each interval and the behavioral complexity represented there also affect the resulting accuracy.

The analysis uses the fixed configuration of Section 4.2.1 so that it extends the LLM comparison rather than introducing another experimental setting. More specifically, all six

LLMs are evaluated with $b = 1$, $g = 3$, $m = 1$, `bge-small-en-v1.5`, top-$k = 10$, no reranker, and the row-level split. For every model, accuracy is calculated in the available 1-5, 6-10, 11-20, 21-30, and 31+ prefix-length buckets of each event log. Empty intervals are omitted. This produces 18 non-empty bucket accuracies per LLM across the four logs.

Figure 4.9 summarizes these earliness results as distributions. Each box contains the 18 bucket-level accuracy values of one model: the central line denotes the median, the box spans the first and third quartiles, the whiskers extend to the most extreme values within 1.5 interquartile ranges, and points beyond them are displayed separately. The values are deliberately not weighted by bucket size, because the purpose of the figure is to compare how consistently each LLM behaves across process stages and logs. Consequently, the box plot complements, rather than replaces, the aggregate accuracy results of RQ1.

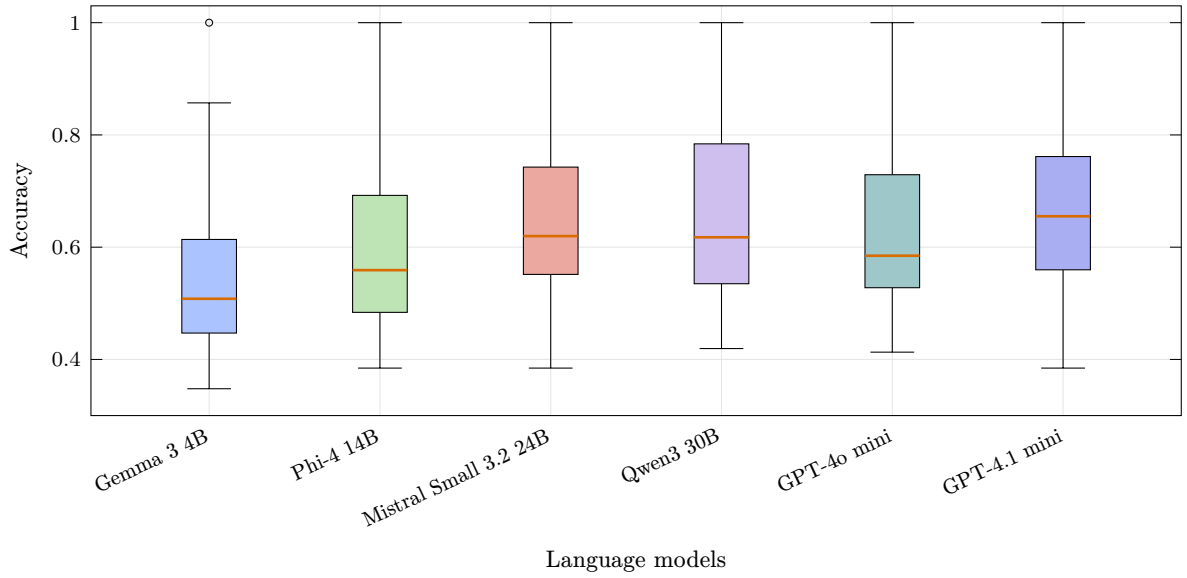

Figure 4.9: Distribution of bucket-level prediction accuracy for the six LLMs across the four event logs under the fixed RQ1 configuration ($b = 1, g = 3, m = 1$) with BGE embeddings. Each model is represented by the 18 non-empty prefix-length bucket accuracies available in the evaluation results.

The distributional comparison reinforces the ranking observed in the aggregate results, while also revealing differences that a single mean cannot show. *GPT-4.1 mini* has the highest median bucket accuracy (0.6551) and a middle 50% range from 0.5597 to 0.7615. It therefore provides the strongest typical performance across the examined stages, although its full range remains broad and confirms that its behavior still depends on the event log and prefix interval. *Mistral Small 3.2 24B* and *Qwen3 30B* have almost identical medians (0.6197 and 0.6176, respectively). Qwen3 nevertheless has the higher upper quartile (0.7841 compared with 0.7427), together with a wider interquartile range, indicating stronger results in favorable buckets but greater variation across stages. This is consistent with its strong aggregate results in RQ1 without implying uniformly superior early prediction.

GPT-4o mini occupies an intermediate position, with a median of 0.5848 and a comparatively compact middle range of 0.5278–0.7291. *Phi-4 14B* follows with a median of 0.5591, while *Gemma 3 4B* has both the lowest median (0.5081) and the lowest first quartile (0.4470). The ordering suggests that the stronger models generally retain their advantage when the evaluation is separated by observed process length, but the overlapping boxes show that no LLM dominates every prefix interval and event log.

The extreme values require particular care. All models reach an accuracy of 1.0 in at least one bucket, but some of these buckets contain very few observations; for example, the 6-10

interval of *BPI Challenge 2012* contains only one evaluated prefix under this configuration. Such values are useful for showing the recorded range, but they do not provide the same evidence as accuracies obtained from the substantially larger middle buckets. Overall, the results show that useful next-activity prediction is already possible from relatively short prefixes, yet earliness is not monotonic: additional observed events do not guarantee higher accuracy. The practical reliability of an early prediction therefore depends jointly on the selected LLM, the structure of the underlying process, and the amount of evidence represented in the relevant prefix-length bucket.

Chapter 5

Conclusion

5.1 Summary

This thesis investigated whether next-activity prediction in Predictive Process Monitoring can benefit from a Retrieval-Augmented Generation pipeline grounded in historical event-log behavior. The starting point of the work was the observation made already in Chapter 1: organizations increasingly need predictive support while cases are still running, yet real event logs are difficult to model because they combine sequential control-flow behavior, event attributes, noise, repeated patterns, and limited generalization to unseen situations. In response to this challenge, the thesis proposed a retrieval-supported in-context learning framework in which process prefixes are transformed into structured textual representations, similar historical prefixes are retrieved through dense similarity search, and the retrieved examples are passed to a Large Language Model to predict the next activity.

The experimental study examined the main components of this pipeline through four research questions. First, the results showed that the choice of LLM has a substantial effect on final predictive quality. Across the four evaluated logs, *GPT-4.1 mini* and *Qwen3 30B* emerged as the strongest overall models, with mean accuracies of 0.6550 and 0.6542, respectively, while Qwen3 achieved the strongest mean class-balanced performance and *GPT-4.1 mini* achieved the strongest typical earliness profile. At the dataset level, *GPT-4.1 mini* performed best on *BPI Challenge 2012* with accuracy 0.6600 and macro-F_1 0.5787, whereas *Qwen3 30B* performed best on *BPI Challenge 2020 International Declarations* and *Hospital Billing*, reaching accuracies of 0.8167 and 0.7167, respectively. The *Sepsis Cases* log was consistently more difficult for all models, which underlines the challenge posed by irregular and heterogeneous healthcare behavior.

Second, the prefix-construction experiments showed that there is no universally optimal representation. The effect of the gap parameter and the number of recent value blocks retained in the textual prefix depended on both the dataset and the downstream LLM. In general, stronger models were better able to exploit richer recent-value retention, while more conservative settings remained preferable in several cases for the baseline local model. This indicates that representation quality in this setting is not only a matter of how much information is exposed, but also of whether the generator can use that additional information effectively.

Third, the retrieval-side experiments showed that the retrieval component is not neutral. The BGE embedding model provided the more stable retrieval reference across the examined settings, while MiniLM remained competitive only in selected cases and was more sensitive to the dataset and to longer textual prefixes. The reranking step produced mixed results: it did not consistently improve top-level accuracy, but in some logs it improved macro-level behavior, suggesting that better second-stage ranking may be more useful for class balance than for dominant-class prediction alone.

Fourth, the stricter generalized evaluation confirmed both the promise and the limits of the approach. When moving from the prefix-level split to the more realistic trace-level split, mean accuracy decreased from 0.6867 to 0.6542, and mean macro-F_1 from 0.5026 to 0.4321. This shows that some of the stronger results under the more favorable protocol do not transfer unchanged to a stricter setting. At the same time, the reduction was not destructive: the framework still preserved meaningful predictive effectiveness across the logs, which supports the central claim of the thesis that retrieval-augmented in-context learning is a viable direction for next-activity prediction.

Taken together, the findings support the main thesis argument. A RAG-based LLM pipeline can produce promising next-activity predictions when it is grounded in similar historical process behavior, but its performance depends materially on the interaction between the language model, the textual representation of prefixes, the retrieval mechanism, and the evaluation protocol. The thesis therefore contributes not only a concrete framework, but also an empirical analysis of where such a framework works well, where it becomes fragile, and which components most strongly shape the final outcome.

5.2 Threats to Validity

Several threats to validity should be considered when interpreting these results. First, the study evaluates a limited set of event logs, even though the selected datasets are diverse and widely used in the literature. The observed behavior of the framework may therefore still depend on the specific control-flow structure, class distribution, attribute richness, and noise characteristics of these logs. Additional validation on more domains and on larger benchmark suites would strengthen the external validity of the findings.

Second, the reported performance is tied to the specific experimental choices made in this thesis, including the prompt design, the prefix textualization scheme, the retrieval depth, the candidate-pool settings, and the selected LLM and embedding models. Since LLM-based systems are sensitive to prompting and retrieval context, alternative but still reasonable configurations could shift the results. This is particularly relevant in a fast-moving area where both proprietary and open-weight models evolve rapidly.

Third, the retrieval space itself introduces an important validity risk. Dense similarity between textualized prefixes does not always correspond to process-relevant similarity. Two prefixes may appear close in embedding space while differing in crucial sequential or attribute-level details, and relevant historical examples may fail to be retrieved. This issue is especially important in the present problem because most embedding models used in the pipeline are trained primarily on natural-language similarity tasks rather than on event-log sequences. As a result, they may capture semantic similarity in a general NLP sense without fully preserving the sequential meaning of prefixes or the operational importance of event values in process data. This limitation may partly explain why retrieval quality, reranking behavior, and richer retained-value settings produced dataset-dependent outcomes.

Fourth, even though the thesis included a stricter trace-level evaluation, generalization remains a central concern. The differences between prefix-level and trace-level performance show that favorable split strategies can overestimate predictive effectiveness. The stricter protocol provides a more realistic view, but it does not eliminate all possible threats related to temporal drift, repeated organizational routines, or future changes in process behavior.

Finally, reproducibility and implementation fidelity are always important concerns in experimental work of this kind. To mitigate this risk, the code used for the experimental pipeline is available online at https://github.com/tzellas/predictive_process_mining_rag. This improves transparency and makes it easier to inspect the preprocessing, retrieval, prompting, and evaluation steps used in the thesis, even though full reproducibility may still depend on access to the same models, APIs, and execution environment.

5.3 Future Work

Several directions follow naturally from the present results. The most immediate extension concerns the retrieval representation itself. Since the embedding models used here are largely optimized for NLP-style semantic similarity rather than for event-log sequences, a promising next step is to fine-tune embedding models specifically for process prefixes so that they better capture sequential meaning, control-flow dependencies, and the predictive role of event values.

This could improve both the quality of the retrieved examples and the downstream usefulness of richer prefix representations.

Future work could also examine stronger retrieval pipelines more broadly, including process-aware reranking strategies, hybrid dense–symbolic retrieval, and retrieval objectives explicitly aligned with next-activity prediction rather than generic semantic similarity. In parallel, the generation side could be extended with additional prompt variants, calibration strategies, or lightweight adaptation methods that preserve the flexibility of the current approach while improving class-balanced robustness.

Another important direction is broader evaluation. The present thesis already showed that stricter split strategies can materially change the conclusions, so future studies should continue to test RAG-based predictive process monitoring under more generalizable protocols, additional datasets, temporal drift settings, and standardized benchmarks. A stronger shared evaluation basis would make it easier to compare LLM-based process prediction methods fairly and to understand when improvements reflect genuine generalization rather than favorable experimental conditions.

Finally, the pipeline could be extended beyond next-activity prediction to related predictive process monitoring tasks such as suffix prediction, outcome prediction, or remaining-time prediction. If retrieval-augmented in-context learning continues to perform well across these tasks, it could become a broader process-aware prediction paradigm rather than a task-specific solution.

Bibliography

[1] C. Di Francescomarino and C. Ghidini, "Predictive Process Monitoring," in *Process Mining Handbook*, W. M. P. van der Aalst and J. Carmona, Eds. Cham, Switzerland: Springer, 2022, pp. 320–346. [Online]. Available: https://doi.org/10.1007/978-3-031-08848-3_10

[2] P. Ceravolo, M. Comuzzi, J. De Weerdt, C. Di Francescomarino, and F. M. Maggi, "Predictive Process Monitoring: Concepts, Challenges, and Future Research Directions," *Process Science*, vol. 1, article 2, 2024. [Online]. Available: https://doi.org/10.1007/s44311-024-00002-4

[3] P. Pfeiffer, L. Abb, P. Fettke, and J.-R. Rehse, "Learning from the Data to Predict the Process: Generalization Capabilities of Next Activity Prediction Algorithms," *Business & Information Systems Engineering*, vol. 67, pp. 357–383, 2025. [Online]. Available: https://doi.org/10.1007/s12599-025-00936-4

[4] D. A. Neu, J. Lahann, and P. Fettke, "A Systematic Literature Review on State-of-the-Art Deep Learning Methods for Process Prediction," *Artificial Intelligence Review*, vol. 55, pp. 801–827, 2022. [Online]. Available: https://doi.org/10.1007/s10462-021-09960-8

[5] E. Rama-Maneiro, J. C. Vidal, and M. Lama, "Deep Learning for Predictive Business Process Monitoring: Review and Benchmark," *IEEE Transactions on Services Computing*, vol. 16, no. 1, pp. 739–756, 2023. [Online]. Available: https://doi.org/10.1109/TSC.2021.3139807

[6] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, "Outcome-Oriented Predictive Process Monitoring: Review and Benchmark," *ACM Transactions on Knowledge Discovery from Data*, vol. 13, no. 2, article 17, pp. 1–57, 2019. [Online]. Available: https://doi.org/10.1145/3301300

[7] C. Di Francescomarino, M. Dumas, F. M. Maggi, and I. Teinemaa, "Clustering-Based Predictive Process Monitoring," *IEEE Transactions on Services Computing*, vol. 12, no. 6, pp. 896–909, Nov.-Dec. 2019. [Online]. Available: https://doi.org/10.1109/TSC.2016.2645153

[8] V. Pasquadibisceglie, A. Appice, G. Castellano, and D. Malerba, "A Multi-View Deep Learning Approach for Predictive Business Process Monitoring," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2382–2395, 2022. [Online]. Available: https://doi.org/10.1109/TSC.2021.3051771

[9] T. H. Nguyen, "Next Process-Activity Prediction Using Switch-Transformer: Approach, Visualization, and Performance Evaluation," *Knowledge and Information Systems*, vol. 67, pp. 11827–11854, 2025. [Online]. Available: https://doi.org/10.1007/s10115-025-02587-z

[10] A. Chiorrini, C. Diamantini, L. Genga, and D. Potena, "Multi-Perspective Enriched Instance Graphs for Next Activity Prediction Through Graph Neural Network," *Journal of Intelligent Information Systems*, vol. 61, pp. 5–25, 2023. [Online]. Available: https://doi.org/10.1007/s10844-023-00777-1

[11] V. Pasquadibisceglie, R. Scaringi, A. Appice, G. Castellano, and D. Malerba, "PROPHET: Explainable Predictive Process Monitoring With Heterogeneous Graph Neural Networks," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 4111–4124, 2024. [Online]. Available: https://doi.org/10.1109/TSC.2024.3463487

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse,

M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[13] A. Berti and M. S. Qafari, "Leveraging Large Language Models (LLMs) for Process Mining (Technical Report)," arXiv preprint arXiv:2307.12701, 2023. [Online]. Available: https://arxiv.org/abs/2307.12701

[14] M. Vidgof, S. Bachhofner, and J. Mendling, "Large Language Models for Business Process Management: Opportunities and Challenges," in *Business Process Management Forum (BPM 2023)*, Lecture Notes in Business Information Processing, vol. 490, Cham, Switzerland: Springer, 2023, pp. 107–123. [Online]. Available: https://doi.org/10.1007/978-3-031-41623-1_7

[15] A. Berti, H. Kourani, and W. M. P. van der Aalst, "PM-LLM-Benchmark: Evaluating Large Language Models on Process Mining Tasks," in *Process Mining Workshops (ICPM 2024)*, Lecture Notes in Business Information Processing, vol. 533, Cham, Switzerland: Springer, 2025, pp. 610–623. [Online]. Available: https://doi.org/10.1007/978-3-031-82225-4_45

[16] M. Grohs, L. Abb, N. Elsayed, and J.-R. Rehse, "Large Language Models Can Accomplish Business Process Management Tasks," in *Business Process Management Workshops (BPM 2023)*, Lecture Notes in Business Information Processing, vol. 492, Cham, Switzerland: Springer, 2024, pp. 453–465. [Online]. Available: https://doi.org/10.1007/978-3-031-50974-2_34

[17] H. Kourani, A. Berti, D. Schuster, and W. M. P. van der Aalst, "Process Modeling with Large Language Models," in *Business Process Modeling, Development and Support*, Lecture Notes in Business Information Processing, vol. 511, Cham, Switzerland: Springer, 2024, pp. 229–244. [Online]. Available: https://doi.org/10.1007/978-3-031-61007-3_18

[18] K. Lashkevich, F. Milani, M. Avramenko, M. Gharleghi, and M. Dumas, "LLM-Assisted Optimization of Waiting Time in Business Processes: A Prompting Method," in *Business Process Management*, pp. 474–492, Cham: Springer, 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-70396-6_27

[19] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html

[20] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2024. [Online]. Available: https://arxiv.org/abs/2312.10997

[21] M. Arslan, H. Ghanem, S. Munawar, and C. Cruz, "A Survey on RAG with LLMs," *Procedia Computer Science*, vol. 246, pp. 3781–3790, 2024. [Online]. Available: https://doi.org/10.1016/j.procs.2024.09.373

[22] A. Casciani, A. Bernardi, E. Marrella, and F. M. Maggi, "Retrieval-Augmented Generation for Next Activity Prediction in Business Process Monitoring," *Information Systems*, vol. 137, p. 102642, 2026. [Online]. Available: https://doi.org/10.1016/j.is.2025.102642

[23] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in *Proceedings of the 17th Conference of the European Chapter of*

the Association for Computational Linguistics, pp. 2014–2037, Dubrovnik, Croatia: Association for Computational Linguistics, 2023. [Online]. Available: https://aclanthology.org/2023.eacl-main.148/

[24] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 2318–2335. [Online]. Available: https://doi.org/10.18653/v1/2024.findings-acl.137

[25] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, Hong Kong, China: Association for Computational Linguistics, 2019. [Online]. Available: https://aclanthology.org/D19-1410/

[26] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," *Advances in Neural Information Processing Systems*, vol. 33, pp. 5776–5788, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[27] F. Klessascheck and L. Pufahl, "Reviewing Uses of Regulatory Compliance Monitoring," *Software and Systems Modeling*, vol. 25, pp. 413–436, 2026. [Online]. Available: https://doi.org/10.1007/s10270-025-01338-6

[28] A. N. Iman, I. M. Kamal, D. Kim, and H. Bae, "Continual Learning Approach With Adaptive Memory Mechanism for Predictive Process Monitoring in Dynamic Business Environments," *IEEE Access*, vol. 13, pp. 206767–206783, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3636194

[29] J. Yuan, D. Grigori, and H. van der Aa, "Enhancing Predictive Process Monitoring Using Semantic Information," in *Process Mining Workshops*, pp. 293–305, Cham: Springer, 2025. [Online]. Available: https://doi.org/10.1007/978-3-031-82225-4_22

[30] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," 2019. [Online]. Available: https://arxiv.org/abs/1901.04085

[31] S. Kaftantzis, A. Bousdekis, G. Theodoropoulou, and G. Miaoulis, "Predictive Business Process Monitoring with AutoML for Next Activity Prediction," *Intelligent Decision Technologies*, vol. 18, no. 3, pp. 1965–1980, 2024. [Online]. Available: https://doi.org/10.3233/IDT-240632

[32] L. Aversano, M. L. Bernardi, M. Cimitile, M. Iammarino, and C. Verdone, "A Data-Aware Explainable Deep Learning Approach for Next Activity Prediction," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106758, 2023. [Online]. Available: https://doi.org/10.1016/j.engappai.2023.106758

[33] N. Mehdiyev, M. Majlatow, and P. Fettke, "Interpretable and Explainable Machine Learning Methods for Predictive Process Monitoring: A Systematic Literature Review," *Artificial Intelligence Review*, vol. 58, no. 12, article 378, 2025. [Online]. Available: https://doi.org/10.1007/s10462-025-11399-0

[34] V. Pasquadibisceglie, A. Appice, G. Castellano, and D. Malerba, "JARVIS: Joining Adversarial Training With Vision Transformers in Next-Activity Prediction," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1593–1606, 2024. [Online]. Available: https://doi.org/10.1109/TSC.2023.3331020

[35] E. Rama-Maneiro, J. C. Vidal, and M. Lama, "Embedding Graph Convolutional Networks in Recurrent Neural Networks for Predictive Monitoring," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 137–151, 2024. [Online]. Available: https://doi.org/10.1109/TKDE.2023.3286017

[36] B. van Dongen, "BPI Challenge 2012," 4TU.ResearchData, 2012. [Online]. Available: https://data.4tu.nl/articles/dataset/BPI_Challenge_2012/12689204/1

[37] B. van Dongen, "BPI Challenge 2020: International Declarations," 4TU.ResearchData, 2020. [Online]. Available: https://doi.org/10.4121/uuid:2bbf8f6a-fc50-48eb-aa9e-c4ea5ef7e8c5

[38] F. Mannhardt, "Sepsis Cases - Event Log," 4TU.ResearchData, 2016. [Online]. Available: https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460

[39] F. Mannhardt, "Hospital Billing - Event Log," 4TU.ResearchData, 2017. [Online]. Available: https://doi.org/10.4121/uuid:76c46b83-c930-4798-a1c9-4be94dfeb741

[40] sentence-transformers, "all-MiniLM-L12-v2," Hugging Face model card. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L12-v2. Accessed: Jul. 9, 2026.

[41] BAAI, "bge-small-en-v1.5," Hugging Face model card. [Online]. Available: https://huggingface.co/BAAI/bge-small-en-v1.5. Accessed: Jul. 9, 2026.

[42] BAAI, "bge-reranker-base," Hugging Face model card. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-base. Accessed: Jul. 9, 2026.

[43] Ollama, "gemma3:4b," Ollama model library. [Online]. Available: https://ollama.com/library/gemma3:4b. Accessed: Jul. 9, 2026.

[44] Ollama, "phi4:14b," Ollama model library. [Online]. Available: https://ollama.com/library/phi4:14b. Accessed: Jul. 9, 2026.

[45] Ollama, "mistral-small3.2:24b," Ollama model library. [Online]. Available: https://ollama.com/library/mistral-small3.2:24b. Accessed: Jul. 9, 2026.

[46] Ollama, "qwen3:30b," Ollama model library. [Online]. Available: https://ollama.com/library/qwen3:30b. Accessed: Jul. 9, 2026.

[47] OpenAI, "GPT-4o mini model documentation," OpenAI Developers. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-4o-mini. Accessed: Jul. 9, 2026.

[48] OpenAI, "GPT-4.1 mini model documentation," OpenAI Developers. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-4.1-mini. Accessed: Jul. 9, 2026.